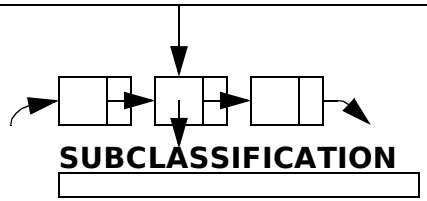
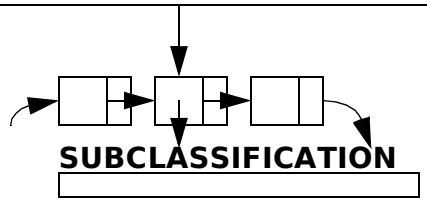
ELEMENT

```
/* Usage */
ELEMENT *element_fetch(
	char *element_name );

/* Process */
char *element_primary_where(
	char *element_name );

char *element_system_string(
	char *ELEMENT_SELECT,
	char *ELEMENT_TABLE,
	char *element_primary_where()
		/* where */ );

ELEMENT *element_parse(
	char *input );

ELEMENT *element_new(
	char *element_name );

ELEMENT *element_calloc( void );

/* Input */
char *element_name;
boolean accumulate_debit;

/* Usage */
LIST *element_subclassification_list(
	char *element_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time );

/* Process */
LIST *subclassification_list = list_new();

LIST *subclassification_element_subclassification_name_list(
	element_primary_where( element_name ),
	SUBCLASSIFICATION_TABLE,
	“display_order” /* order_column */ );

for subclassification_name in
	subclassification_element_subclassification_name_list()
{
	list_set(
		subclassification_list,
		subclassification_element_fetch(
			subclassification_name,
			begin_transaction_date_time,
			end_transaction_date_time ) );
}

/* Usage */
LIST *element_account_list(
	LIST *subclassification_list );

/* Process */
LIST *account_list = list_new();

for SUBCLASSIFICATION *subclassification in subclassification_list
{
	list_set_list(
		account_list,
		subclassification->account_list );
}

LIST *account_balance_sort_list( account_list );
```



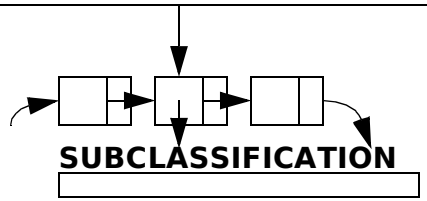
ELEMENT_FUND

```
/* Usage */
ELEMENT_FUND *element_fund_fetch(
	char *element_name,
	char *fund_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

/* Process */
char *element_primary_where(
	element_name );

char *element_system_string(
	ELEMENT_SELECT,
	ELEMENT_TABLE,
	element_primary_where()
		/* where */ );

ELEMENT_FUND *element_fund_parse(
	char *input,
	char *fund_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

ELEMENT_FUND *element_fund_new(
	char *element_name,
	char *fund_name );

ELEMENT_FUND *element_fund_calloc( void );

/* Input */
char *element_name;
char *fund_name;
boolean accumulate_debit;
```



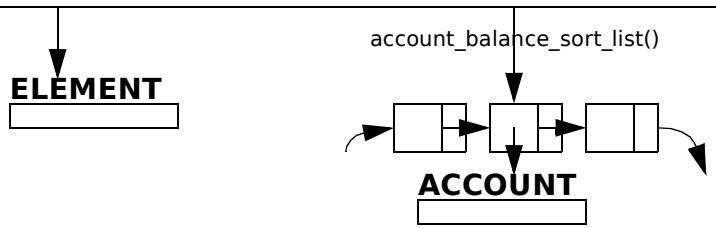
ELEMENT_PROGRAM

```
/* Usage */
ELEMENT_PROGRAM *element_program_fetch(
	char *element_name,
	char *program_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

/* Process */
char *element_primary_where(
	element_name );

char *element_system_string(
	ELEMENT_SELECT,
	ELEMENT_TABLE,
	element_primary_where()
		/* where */ );

ELEMENT_PROGRAM *element_program_parse(
	char *input,
	char *program_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time,
	boolean fetch_subclassification_list,
	boolean fetch_account_list,
	boolean fetch_journal_list );

ELEMENT_PROGRAM *element_program_new(
	char *element_name,
	char *program_name );

ELEMENT_PROGRAM *element_program_calloc( void );

/* Input */
char *element_name;
char *program_name;
boolean accumulate_debit;
```



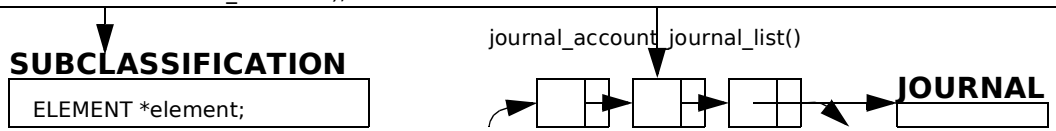
SUBCLASSIFICATION

```
/* Usage */
SUBCLASSIFICATION *subclassification_fetch(
   char *subclassification_name,
   boolean fetch_element );

/* Process */
char *subclassification_primary_where(
   char *subclassification_name );

char *subclassification_system_string(
   char *SUBCLASSIFICATION_SELECT,
   char *SUBCLASSIFICATION_TABLE,
   char *subclassification_primary_where()
         /* where */ );

SUBCLASSIFICATION *subclassification_parse(
   char *input,
   boolean fetch_element );

SUBCLASSIFICATION *subclassification_new(
   char *subclassification_name );

SUBCLASSIFICATION *subclassification_calloc(
   void );

/* Input */
char *subclassification_name;
char *element_name;
int display_order;

if ( fetch_element )
{
   ELEMENT *element =
         element_fetch(
               subclassification->element_name );
}

/* Usage */
SUBCLASSIFICATION *subclassification_element_fetch(
   char *subclassification_name,
   char *begin_transaction_date_time,
   char *end_transaction_date_time );

/* Process */
SUBCLASSIFICATION *subclassification =
   subclassification_new(
         subclassification_name );

LIST *account_subclassification_account_name_list(
   subclassification_primary_where(
         subclassification_name ),
   ACCOUNT_TABLE );

if ( !list_rewind(
   account_subclassification_account_name_list() )
{
   return (SUBCLASSIFICATION *)0;
}

LIST *account_list = list_new();

for account_name in
   account_subclassification_account_name_list()
{
   list_set(
         account_list,
         account_subclassification_fetch(
               account_name,
               begin_transaction_date_time,
               end_transaction_date_time ) );
}

LIST *account_balance_sort_list(
   account_list );

/* Public */
LIST *
   subclassification_element_subclassification_name_list(
         char *element_primary_where(),
         char *SUBCLASSIFICATION_TABLE,
         char *order_column );
```



ACCOUNT (1)

```
/* Usage */
ACCOUNT *account_fetch(
      char *account_name,
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
char *account_primary_where(
      char *account_name );

char *account_system_string(
      char *ACCOUNT_SELECT,
      char *ACCOUNT_TABLE,
      char *account_primary_where()
            /* where */ );

ACCOUNT *account_parse(
      char *input,
      boolean fetch_subclassification,
      boolean fetch_entity );

ACCOUNT *account_new( char *account_name );

ACCOUNT *account_calloc( void );

/* Input */
char *account_name;
char *subclassification_name;
char *hard_coded_account_key;
int chart_account_number;
int annual_budget;

if ( fetch_subclassification )
{
      SUBCLASSIFICATION *subclassification =
            subclassification_fetch(
                  subclassification_name,
                  fetch_entity );
}

/* Attributes */
double balance;
double payment_amount;
LIST *journal_account_journal_list;
int percent_of_asset;
int percent_of_revenue;
int delta_prior;
LIST *following_balance_zero_journal_list;
LIST *liability_journal_list;
double liability_due;
char *action_string;

/* Usage */
ACCOUNT *account_key_fetch(
      char *hard_coded_account_key,
      boolean fetch_subclassification,
      boolean fetch_entity );

/* Process */
char *account_hard_coded_key_where(
      char *hard_coded_account_key );

ACCOUNT *account_parse(
      string_pipe(
            account_system_string(
                  ACCOUNT_SELECT,
                  ACCOUNT_TABLE,
                  account_hard_coded_key_where(
                        hard_coded_account_key ) ) ),
      fetch_subclassification,
      fetch_entity );

/* Usage */
LIST *account_system_list(
      char *account_system_string(),
      boolean fetch_subclassification,
      boolean fetch_element );

/* Usage */
LIST *account_list(
      char *where,
      boolean fetch_subclassification,
      boolean fetch_element );

/* Process */
char *account_system_string(
      ACCOUNT_SELECT,
      ACCOUNT_TABLE,
      where );

LIST *account_system_list(
      account_system_string(),
      fetch_subclassification,
      fetch_element );

/* Usage */
ACCOUNT *account_subclassification_fetch(
      char *account_name,
      char *begin_transaction_date_time,
      char *end_transaction_date_time );

/* Process */
ACCOUNT *account =
      account_new(
            account_name );

LIST *journal_account_journal_list(
      account_name,
      begin_transaction_date_time,
      end_transaction_date_time );

/* Usage */
boolean account_accumulate_debit(
      char *account_name,
      boolean expect_lots );

/* Process */
if ( expect_lots )
{
      static LIST *list = account_list( “1 = 1”, 0, 0 );

      ACCOUNT *account =
            account_seek(
                  account_name,
                  list );
      return
      subclassification->element->accumulate_debit;
}
else
{
      ACCOUNT *account =
            account_fetch(
                  account_name,
                  1 /* fetch_subclassification */,
                  1 /* fetch_element */ );

      return
      subclassification->element->accumulate_debit;
}
```

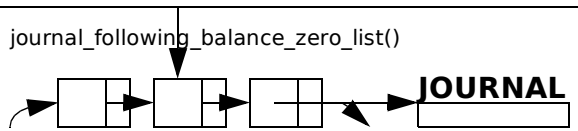


ACCOUNT (2)

```
/* Usage */
LIST *account_balance_sort_list(
	LIST *account_list );

/* Process */
int account_balance_match_function(
	ACCOUNT *account_from_list,
	ACCOUNT *account_compare );

JOURNAL *from_list_latest_journal =
	list_last(
		account_from_list->
			journal_account_journal_list() );

JOURNAL *compare_latest_journal =
	list_last(
		account_compare->
			journal_account_journal_list() );

if ( !from_list_latest_journal
||	!from_list_latest_journal->balance )
{
	return 1;
}

if ( !compare_latest_journal
||	!compare_latest_journal->balance )
{
	return 1;
}

if (	from_list_latest_journal->balance <=
	compare_latest_journal->balance )
{
	return 1;
}
else
{
	return -1;
}

/* Usage */
LIST *account_list_delta_prior_set(
	LIST *prior_account_list,
	LIST *account_list );

/* Process */
void account_delta_prior_set(
	LIST *prior_account_list,
	ACCOUNT *account );

ACCOUNT *prior_account =
	account_seek(
		account->account_name,
		prior_account_list );

JOURNAL *prior_latest_journal =
	journal_list_latest(
		prior_account->
			journal_account_journal_list );

JOURNAL *latest_journal =
	journal_list_latest(
		account->
			journal_account_journal_list );

int account_delta_prior(
	double prior_latest_journal->balance,
	double latest_journal->balance );

/* Usage */
LIST *account_list_percent_of_asset_set(
	LIST *account_list,
	double asset_total );

/* Process */
for ACCOUNT *account in account_list
{
	account->percent_of_asset =
		float_percent_of_total(
			account_balance(
				account->
					journal_account_journal_list ),
			asset_total );
}

/* Usage */
LIST *account_list_percent_of_revenue_set(
	LIST *account_list,
	double revenue_total );

/* Process */
for ACCOUNT *account in account_list
{
	account->percent_of_revenue =
		float_percent_of_total(
			account_balance(
				account->
					journal_account_journal_list ),
			revenue_total );
}
```

ACCOUNT (3)

```
/* Usage */
double account_balance(
	LIST *account->journal_account_journal_list );

/* Process */
JOURNAL *latest_journal =
	journal_list_latest(
		journal_account_journal_list );

return latest_journal->balance;

/* Usage */
double account_debit_total(
	LIST *account_list );

/* Process */
for ACCOUNT *account in account_list
{
	JOURNAL *latest_journal =
		journal_list_latest(
			account->journal_account_journal_list );

	if ( !account_accumulate_debit(
		account->account_name,
		1 /* expect_lots */ )
	&& latest_journal->balance < 0.0 )
	{
		debit_total += -latest_journal->balance;
	}
	else
	if ( account_accumulate_debit(
		account->account_name,
		1 /* expect_lots */ )
	&& latest_journal->balance > 0.0 )
	{
		debit_total += latest_journal->balance;
	}
}

/* Usage */
double account_credit_total(
	LIST *account_list );

/* Process */
for ACCOUNT *account in account_list
{
	JOURNAL *latest_journal =
		journal_list_latest(
			account->journal_account_journal_list );

	if ( account_accumulate_debit(
		account->account_name,
		1 /* expect_lots */ )
	&& latest_journal->balance < 0.0 )
	{
		credit_total += -latest_journal->balance;
	}
	else
	if ( !account_accumulate_debit(
		account->account_name,
		1 /* expect_lots */ )
	&& latest_journal->balance > 0.0 )
	{
		credit_total += latest_journal->balance;
	}
}

/* Usage */
double account_balance_total(
	LIST *account_list );

/* Process */
for ACCOUNT *account in account_list
{
	JOURNAL *latest_journal =
		journal_list_latest(
			account->journal_account_journal_list );

	balance_total += latest_journal->balance;
}

/* Usage */
char *account_hard_coded_account_name(
	char *account_key,
	boolean warning_only,
	const char *calling_function_name );

/* Process */
ACCOUNT *account_key_seek(
	char *account_key,
	static LIST *account_list() );
```



ACCOUNT (4)

```
/* Usage */
void account_list_action_string_set(
        LIST *account_list,
        char *application_name,
        char *session_key,
        char *login_name,
        char *role_name,
        char *beginning_date,
        char *as_of_date );

/* Process */
char *account_action_string(
        char *application_name,
        char *session_key,
        char *login_name,
        char *role_name,
        char *begin_date_string,
        char *as_of_date,
        char *account_name );

/* Public */
char *account_name_format(
        char *account_name );

char *account_name_display(
        char *account_name );

ACCOUNT *account_seek(
        char *account_name,
        LIST *account_list() );

char *account_depreciation_expense( void );
char *account_sales_tax_payable( void );
char *account_accumulated_depreciation( void );
char *account_revenue( void );
char *account_receivable( void );
char *account_uncleared_checks( void );
char *account_shipping_revenue( void );
char *account_cash( void );
char *account_gain( void );
char *account_loss( void );
char *account_payable( void );
char *account_fees_expense( void );

LIST *account_cash_name_list(
        char *ACCOUNT_TABLE,
        char *SUBCLASSIFICATION_CASH );

LIST *account_current_liability_name_list(
        char *ACCOUNT_TABLE,
        char *SUBCLASSIFICATION_CURRENT_LIABILITY,
        char *ACCOUNT_UNCLEARED_CHECKS );

LIST *account_receivable_name_list(
        char *ACCOUNT_TABLE,
        char *SUBCLASSIFICATION_RECEIVABLE );

ACCOUNT *account_getset(
        LIST *account_list,
        char *account_name );
```

JOURNAL (1)

```
/* Usage */
LIST *journal_year_list(
	int tax_year,
	char *tax_form_line_account->account_name );

/* Process */
char *journal_year_where(
	int tax_year,
	char *account_name,
	char *PREDICTIVE_PRECLOSE_TIME );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_year_where() ),
	0 /* not fetch_check_number */,
	1 /* fetch_memo */ );

/* Usage */
LIST *journal_list(
	char *full_name,
	char *street_address,
	char *transaction_date_time );

/* Process */
char *transaction_primary_where(
	full_name,
	street_address,
	transaction_date_time );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		transaction_primary_where() ),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
LIST *journal_minimum_list(
	char *minimum_transaction_date_time,
	char *account_name );

/* Process */
char *journal_minimum_where(
	char *minimum_transaction_date_time,
	char *account_name );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_minimum_where() ),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
LIST *journal_entity_list(
	char *full_name,
	char *street_address,
	char *account_name );

/* Process */
char *journal_entity_where(
	char *full_name,
	char *street_address,
	char *account_name );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_entity_where() ),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
JOURNAL *journal_account_fetch(
	char *transaction_date_time,
	char *account_name );

/* Process */
char *journal_transaction_account_where(
	char *transaction_date_time,
	char *account_name );

JOURNAL *journal_parse(
	string_pipe(
		journal_system_string(
			JOURNAL_SELECT,
			JOURNAL_TABLE,
			journal_transaction_account_where() ),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
void journal_propagate(
	char *transaction_date_time,
	char *account_name );

/* Process */
JOURNAL *journal_prior(
	transaction_date_time,
	account_name );

LIST *journal_list_prior(
	journal_prior(),
	account_name );

LIST *journal_list_set_balance(
	LIST *journal_list_prior() /* in/out */,
	boolean account_accumulate_debit(
		account_name,
		0 /* not expect_lots */ ) );

void journal_list_update(
	journal_list_set_balance() );

/* Usage */
void journal_list_update(
	LIST *journal_list_set_balance() );

/* Process */
char *journal_update_system_string(
	char *JOURNAL_TABLE );

FILE *update_pipe =
	popen( journal_update_system_string(), “w” );

for JOURNAL *journal in journal_list_set_balance
{
	fprintf( update_pipe, journal->transaction_count );
	fprintf( update_pipe, journal->previous_balance );
	fprintf( update_pipe, journal->balance );
}

pclose( update_pipe );
```

JOURNAL (2)

```
/* Usage */
void journal_account_name_list_propagate(
	char *transaction_date_time,
	LIST *account_name_list );

/* Process */
for account_name in account_name_list
{
	journal_propagate(
		transaction_date_time,
		account_name );
}

/* Usage */
LIST *journal_system_list(
	char *journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_transaction_where() ),
	boolean fetch_check_number,
	boolean fetch_memo );

/* Process */
LIST *list = list_new();
FILE *pipe = popen( journal_system_string(), “r” );

for input in string_input( pipe )
{
	list_set(
		list,
		journal_parse() );
}

/* Usage */
JOURNAL *journal_parse(
	char *input,
	boolean fetch_check_number,
	boolean fetch_memo );

JOURNAL *journal_new(
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	char *account_name );

/* Attributes */
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double previous_balance;
double debit_amount;
double credit_amount;
double balance;
int transaction_count;
int check_number;
char *memo;
boolean match_sum_taken;
double journal_amount;

if ( fetch_check_number )
{
	int check_number =
		atoi(
			journal_transaction_cell_fetch(
				full_name,
				street_address,
				transaction_date_time,
				“check_number” ) );
}
else
if ( fetch_memo )
{
	char *memo =
		journal_transaction_cell_fetch(
			full_name,
			street_address,
			transaction_date_time,
			“memo” );
}

/* Usage */
char *journal_transaction_cell_fetch(
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	char *attribute_name );

/* Process */
char *journal_transaction_system_string(
	attribute_name,
	TRANSACTION_TABLE,
	transaction_primary_where() );

char *string_input(
	journal_transaction_system_string() );

/* Usage */
JOURNAL *journal_prior(
	char *transaction_date_time,
	char *account_name );

/* Process */
char *journal_max_prior_where(
	char *transaction_date_time,
	char *account_name );

char *journal_max_prior_transaction_date_time(
	char *journal_max_prior_where(),
	char *JOURNAL_TABLE );

JOURNAL *journal_account_fetch(
	journal_max_prior_transaction_date_time()
		/* transaction_date_time */,
	account_name );

/* Usage */
JOURNAL *journal_latest(
	char *transaction_date_time_closing,
	char *account_name );

/* Process */
char *journal_max_where(
	char *transaction_date_time_closing,
	char *account_name );

char *journal_max_transaction_date_time(
	char *journal_max_where(),
	char *JOURNAL_TABLE );

JOURNAL *journal_account_fetch(
	journal_max_transaction_date_time()
		/* transaction_date_time */,
	account_name );
```

JOURNAL (3)

```
/* Usage */
LIST *journal_account_journal_list(
	char *account_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time );

/* Process */
char *journal_account_where(
	char *account_name,
	char *begin_transaction_date_time,
	char *end_transaction_date_time );

LIST *journal_system_list(
	journal_system_string(
		JOURNAL_SELECT,
		JOURNAL_TABLE,
		journal_account_where() ),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
LIST *journal_POR_list(
	char *account_name );

/* Process */
char *account_primary_where(
	account_name );

char *journal_system_string(
	JOURNAL_SELECT,
	JOURNAL_TABLE,
	account_primary_where() );

LIST *journal_system_list(
	journal_system_string(),
	0 /* not fetch_check_number */,
	0 /* not fetch_memo */ );

/* Usage */
LIST *journal_following_balance_zero_list(
	char *account_name );

/* Process */
char *journal_following_latest_zero_balance_transaction_date_time(
	char *account_name,
	char *JOURNAL_TABLE );

LIST *journal_minimum_list(
	journal_following_latest_zero_balance_transaction_date_time()
		/* minimum_transaction_date_time */,
	account_name );

/* Usage */
LIST *journal_list_prior(
	JOURNAL *journal_prior(),
	char *account_name );

/* Process */
if ( journal_prior )
{
	LIST *journal_list =
		journal_list_minimum(
			journal_prior->transaction_date_time
				/* minimum_transaction_date_time */,
			account_name );
}
else
{
	LIST *journal_list =
		journal_POR_list(
			account_name );

	JOURNAL *first_journal = list_first( journal_list );
	first_journal->previous_balance = 0.0;
	first_journal->transaction_count = 1;
}

if ( journal_prior && list_length( journal_list ) == 1 )
{
	journal_list = (LIST *)0;
}
```

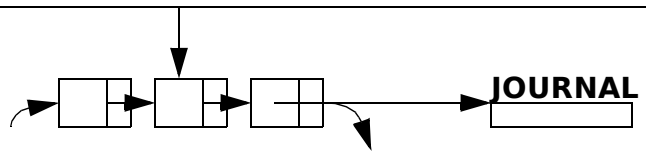
JOURNAL (4)

```
/* Private */
char *journal_system_string(
	char *JOURNAL_SELECT,
	char *JOURNAL_TABLE,
	char *where );

char *journal_transaction_system_string(
	char *attribute_name,
	char *JOURNAL_TRANSACTION_TABLE,
	char *where );

/* Public */
JOURNAL *journal_list_latest(
	LIST *journal_list );

double journal_credit_debit_difference_sum(
	LIST *journal_list );

double journal_debit_credit_difference_sum(
	LIST *journal_list );
```

JOURNAL_PROPERTY

TRANSACTION (1)

```
/* Usage */
LIST *transaction_list(
    char *where,
    boolean fetch_journal_ledger );

/* Process */
char *transaction_system_string(
    char *TRANSACTION_SELECT,
    char *TRANSACTION_TABLE,
    char *where );

FILE *input_pipe =
    popen(
        transaction_system_string(),
        "r" );

LIST *list = list_new();

for input in string_input()
{
    list_set(
        list,
        transaction_parse(
            input,
            fetch_journal_ledger ) );
}

/* Usage */
void transaction_fetch_update(
    char *full_name,
    char *street_address,
    char *transaction_date_time );

/* Process */
TRANSACTION *transaction =
    transaction_fetch(
        full_name,
        street_address,
        transaction_date_time,
        1 /* fetch_journal_ledger */ );

double transaction_amount(
    LIST *transaction->journal_list );

char *transaction_feeder_account(
    LIST *transaction->journal_list );

char *transaction_memo(
    char *transaction->memo );

void transaction_update(
    char *full_name,
    char *street_address,
    char *transaction_date_time,
    double transaction_amount(),
    char *transaction_feeder_account(),
    char *transaction_memo(),
    int check_number,
    boolean lock_transaction,
    LIST *journal_list );

/* Usage */
TRANSACTION *transaction_fetch(
    char *full_name,
    char *street_address,
    char *transaction_date_time,
    boolean fetch_journal_ledger );

/* Process */
char *transaction_primary_where(
    char *full_name,
    char *street_address,
    transaction_date_time );

char *transaction_system_string(
    char *TRANSACTION_SELECT,
    char *TRANSACTION_TABLE,
    char *transaction_primary_where() );

TRANSACTION *transaction_parse(
    char *string_pipe_fetch(
        transaction_system_string() )
            /* input */,
    boolean fetch_journal_ledger );

TRANSACTION *transaction_new(
    full_name,
    street_address,
    transaction_date_time );

TRANSACTION *transaction_calloc(
    void );

/* Attributes */
char *full_name;
char *street_address;
char *transaction_date_time;
double transaction_amount;
char *memo;
int check_number;
boolean lock_transaction;
LIST *journal_list;
```



TRANSACTION (2)

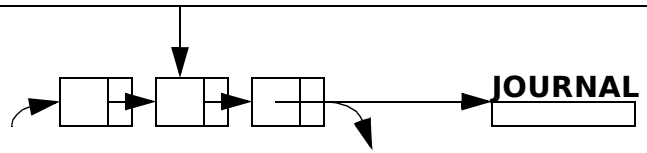
```
/* Usage */
void transaction_update(
	char *full_name,
	char *street_address,
	char *transaction_date_time,
	double transaction_amount(),
	char *transaction_feeder_account(),
	char *transaction_memo(),
	int check_number,
	boolean lock_transaction,
	LIST *journal_list );

/* Process */

/* Usage */
/* Returns inserted transaction_date_time */
char *transaction_insert(
	full_name,
	street_address,
	transaction_date_time,
	transaction_amount,
	memo,
	check_number,
	lock_transaction );

/* Process */
FILE *transaction_insert_pipe_open(
	void );

/* Returns inserted transaction_date_time */
char *transaction_insert_pipe(
	insert_pipe,
	full_name,
	street_address,
	transaction_date_time,
	transaction_amount,
	memo,
	check_number,
	lock_transaction,
	journal_list );

void transaction_check_insert(
	insert_pipe,
	check_number );

void transation_lock_insert(
	insert_pipe,
	lock_transaction );

/* Usage */
/* Returns transaction_list */
LIST *transaction_list_insert(
	transaction_list );

/* Process */
void transaction_list_journal_insert(
	transaction_list );

/* Usage */
/* Performs journal_propagate() */
void transaction_delete(
	full_name,
	street_address,
	transaction_date_time );

/* Public */
TRANSACTION *transaction_seek(
	transaction_list,
	full_name,
	street_address,
	transaction_date_time );

/* Returns race-free transaction_date_time */
char *transaction_race_free_fetch(
	transaction_date_time );

TRANSACTION *transaction_binary(
	full_name,
	street_address,
	transaction_date_time,
	double transaction_amount,
	char *memo,
	char *debit_account,
	char *credit_account );

char *transaction_memo(
	memo );

boolean transaction_exists(
	transaction_date_time );

char *transaction_display(
	transaction );

TRANSACTION *transaction_check_seek(
	transaction_list,
	check_number );

boolean transaction_closing_entry_exists(
	char *transaction_date );

char *transaction_date_time_closing(
	char *transaction_date,
	boolean preclose_time,
	boolean transaction_closing_entry_exists() );

boolean transaction__increase(
	LIST *journal_list,
	LIST *account_cash_list(),
	LIST *account_current_liability_list() );
```

TRANSACTION_AMOUNT

```
/* Usage */
TRANSACTION_AMOUNT *transaction_amount_new(
	LIST *journal_list,
	LIST *account_cash_list(),
	LIST *account_current_liability_list() );
```

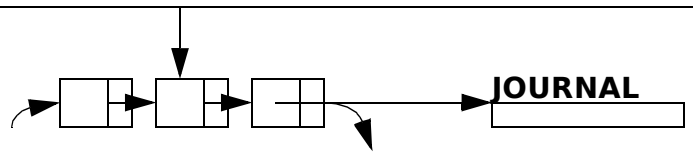
TRANSACTION_PROGRAM

```
/* Usage */
TRANSACTION *transaction_program_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *program_name,
        boolean fetch_journal_ledger );

/* Process */
```



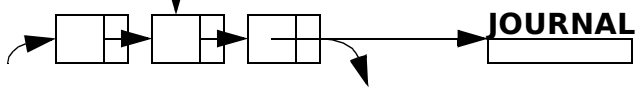
TRANSACTION_FUND

```
/* Usage */
TRANSACTION *transaction_program_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *fund_name,
        boolean fetch_journal_ledger );

/* Process */
```



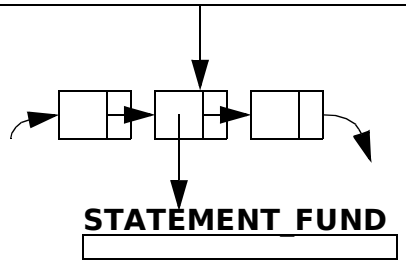
TRANSACTION_PROPERTY

```
/* Usage */
TRANSACTION *transaction_property_fetch(
        full_name,
        street_address,
        transaction_date_time,
        char *property_street_address,
        boolean fetch_journal_ledger );

/* Process */
```



STATEMENT (1)

```
/* Usage */
STATEMENT *statement_new(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    char *process_name,
    boolean exists_logo_filename,
    LIST *filter_element_name_list,
    char *as_of_date,
    int prior_year_count,
    LIST *fund_name_list,
    char *subclassification_option_string,
    char *fund_aggregation_string,
    char *output_medium_string );

STATEMENT *statement_steady_state(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    char *process_name,
    boolean exists_logo_filename,
    LIST *filter_element_name_list,
    char *as_of_date,
    int prior_year_count,
    LIST *fund_name_list,
    char *subclassification_option_string,
    char *fund_aggregation_string,
    char *output_medium_string,
    boolean with_postclose,
    STATEMENT *statement );

enum subclassification_option
    statement_subclassification_option(
        subclassification_option_string );

enum fund_aggregation
    statement_fund_aggregation(
        fund_name_list,
        fund_aggregation_string );

enum output_medium
    statement_output_medium(
        output_medium_string );

LIST *statement_fund_name_list( void );

char *statement_begin_date_string(
    char *as_of_date );

char *statement_title(
    char *application_name,
    boolean exists_logo_filename,
    char *process_name );

char *statement_subtitle(
    statement_begin_date_string(),
    char *as_of_date,
    enum fund_aggregation );

LIST *statement_fund_list(
    application_name,
    session,
    login_name,
    role_name,
    filter_element_name_list,
    statement_begin_date_string(),
    as_of_date,
    prior_year_count,
    fund_name_list,
    char *progam_name,
    statement_subclassification_option(),
    statement_fund_aggregation(),
    boolean with_postclose );

boolean statement_exists_closing_entry(
    char *as_of_date );

void statement_account_delta_list(
    char *account_name,
    LIST *prior_year_list );

void statement_html_account_delta_list(
    char *account_name,
    LIST *prior_year_list );

void statement_subclassification_delta_list(
    char *subclassification_name,
    LIST *prior_year_list );

void statement_html_subclassification_delta_list(
    char *subclassification_name,
    LIST *prior_year_list );

void statement_PDF_account_delta_list(
    char *account_name,
    LIST *prior_year_list );

void statement_PDF_subclassification_delta_list(
    char *subclassification_name,
    LIST *prior_year_list );

int statement_delta_prior(
    double prior_total,
    double total );
```



statement_subclassification_option

subclassification_aggregate,
subclassification_display,
subclassification_omit

statement_fund_aggregation

single_fund,
sequential,
consolidated,
no_fund

statement_output_medium

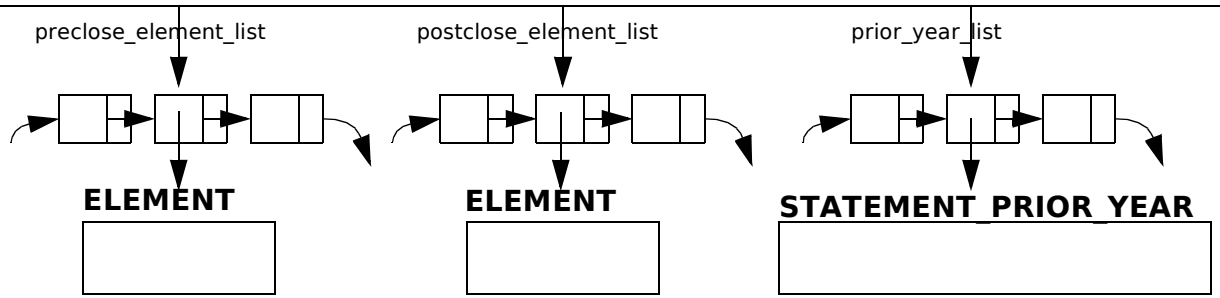
output_table,
output_PDF
output_spreadsheet,
output_stdout

STATEMENT (2)

```
/* Process */
SUBCLASSIFICATION *statement_net_income_subclassification(
        double *equity_element_current_balance,
        int *equity_element_percent_of_asset,
        SUBCLASSIFICATION *last_subclassification,
        EQUITY_ELEMENT *equity_element,
        char *fund_name,
        char *transaction_date_time_closing,
        double asset_element_current_balance,
        boolean is_financial_position );

ELEMENT *statement_equity_liability_element(
        double equity_element_current_balance,
        double asset_element_current_balance,
        double liability_element_current_balance );
```

STATEMENT_FUND (1)

```
/* Usage */
STATEMENT_FUND *statement_fund_fetch(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    LIST *filter_element_name_list,
    char *begin_date_string,
    char *as_of_date,
    int prior_year_count,
    char *fund_name,
    enum subclassification_option,
    boolean with_postclose );

/* Process */
char *transaction_date_time_nominal =
    transaction_date_time_closing(
        as_of_date,
        1 /* preclose_time_boolean */,
        transaction_closing_entry_exists(
            as_of_date ) );

char *transaction_date_time_fixed =
    transaction_date_time_closing(
        as_of_date,
        0 /* not preclose_time_boolean */,
        transaction_closing_entry_exists(
            as_of_date ) );

LIST *statement_fund_element_list(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    LIST *filter_element_name_list,
    char *begin_date_string,
    char *transaction_date_time_nominal,
    char *transaction_date_time_fixed,
    char *fund_name,
    enum subclassification_option );

LIST *preclose_element_list =
    statement_fund_element_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        begin_date_string,
        transaction_date_time_nominal,
        transaction_date_time_nominal,
        fund_name,
        subclassification_option );

LIST *postclose_element_list =
    statement_fund_element_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        begin_date_string,
        transaction_date_time_fixed,
        transaction_date_time_fixed,
        fund_name,
        subclassification_option );

LIST *prior_year_list;

for years_ago = 1 to prior_year_count
{
    list_set(
        prior_year_list,
        statement_prior_year_new(
            application_name,
            session,
            login_name,
            role_name,
            filter_element_name_list,
            as_of_date,
            years_ago,
            fund_name,
            subclassification_option ) );
}

void statement_fund_list_steady_state(
    LIST *statement_fund_list,
    char *title,
    char *subtitle,
    boolean exists_logo_filename );

STATEMENT_FUND *statement_fund_steady_state(
    LIST *preclose_element_list,
    LIST *postclose_element_list,
    char *title,
    char *subtitle,
    boolean exists_logo_filename,
    STATEMENT_FUND *statement_fund );

void statement_fund_list_prior_year_list_set(
    LIST *statement_fund_list,
    int prior_year_count );

LIST *statement_fund_steady_state_prior_year_list(
    int prior_year_count,
    LIST *preclose_element_list,
    STATEMENT_FUND *statement_fund );

STATEMENT_FUND *statement_fund_seek(
    char *fund_name,
    LIST *statement_fund_list );
```

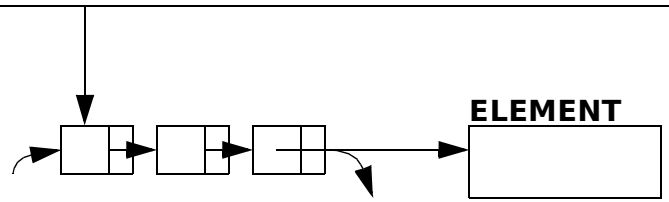


STATEMENT_FUND (2)

```
/* Process */
double preclose_debit_total =
    element_list_debit_total(
        preclose_element_list );

double preclose_credit_total =
    element_list_credit_total(
        preclose_element_list );

double postclose_debit_total =
    element_list_debit_total(
        postclose_element_list );

double postclose_credit_total =
    element_list_credit_total(
        postclose_element_list );

boolean statement_fund_exists_postclose(
    LIST *statement_fund_list );

void statement_fund_list_net_income_set(
    LIST *statement_fund_list );

void statement_fund_list_net_income_fetch(
    LIST *statement_fund_list );

double statement_fund_net_income_fetch(

boolean  statement_fund_exists_prior_year(
    LIST *statement_fund_list );

double statement_fund_total(
    LIST *subclassification_list,
    LIST *account_list );

double revenue_total =
    element_seek(
        ELEMENT_REVENUE,
        preclose_element_list )->
            element_current_balance() );

double expense_total =
    element_seek(
        ELEMENT_EXPENSE,1
        preclose_element_list )->
            element_current_balance() );

double gain_total =
    element_seek(
        ELEMENT_GAIN,
        preclose_element_list )->
            element_current_balance() );

double loss_total =
    element_seek(
        ELEMENT_LOSS,
        preclose_element_list )->
            element_current_balance() );

double net_income =
    statement_fund_net_income(
        revenue_total,
        expense_total,
        gain_total,
        loss_total );

int statement_fund_net_income_percent(
    statement_fund_net_income(),
    revenue_total );

char *statement_fund_caption(
    char *statement_title(),
    char *statement_subtitle(),
    char *statement_fund->fund_name,
    boolean exists_logo_filename );

void statement_fund_list_equity_set(
    LIST *statement_fund_list,
    boolean is_financial_position,
    enum subclassification_option );

void statement_fund_display_equity(
    double *element_current_balance,
    int *percent_of_asset,
    element->subclassification_list /* out */,
    element->equity_element /* in/out */,
    double net_income,
    double asset_current_balance,
    boolean is_financial_position );

void statement_fund_omit_equity(
    double *element_current_balance,
    int *percent_of_asset,
    LIST *account_list /* out */,
    element->equity_element /* in/out */,
    double net_income,
    double asset_current_balance,
    boolean is_financial_position );

void statement_fund_aggregate_equity(
    double *element_current_balance,
    int *percent_of_asset,
    LIST *subclassification_list /* out */,
    element->equity_element /* in/out */,
    double net_income,
    double asset_current_balance,
    boolean is_financial_position );
```

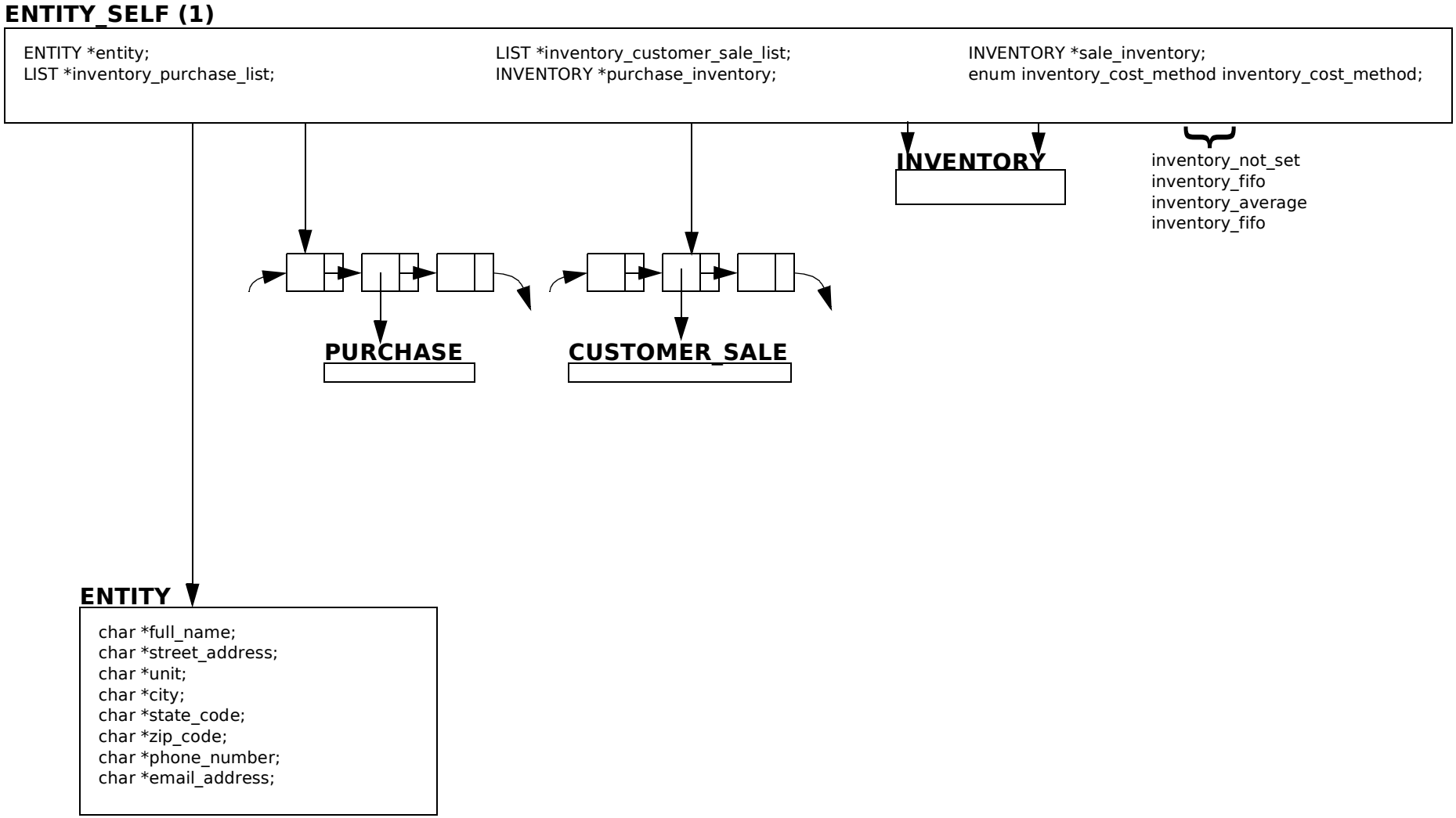
STATEMENT_PRIOR_YEAR

```
/* Usage */
STATEMENT_PRIOR_YEAR *
    statement_prior_year_new(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        LIST *filter_element_name_list,
        char *as_of_date,
        int years_ago,
        char *fund_name,
        enum subclassification_option );

/* Attributes */
char *application_name;
char *session;
char *login_name;
char *role_name;
LIST *filter_element_name_list;
char *as_of_date;
int years_ago;
char *fund_name;
enum subclassification_option subclassification_option;

/* Process */
char *prior_date_string =
    strdup(
        date_yyyy_mm_dd_display(
            date_subtract_year(
                date_yyyy_mm_dd_new(
                    as_of_date ),
                years_ago ) ) );

char *transaction_date_time_nominal =
    transaction_date_time_closing(
        prior_date_string,
        1 /* preclose_time_boolean */,
        transaction_closing_entry_exists(
            prior_date_string ) );

char *transaction_date_time_fixed =
    transaction_date_time_closing(
        prior_date_string,
        0 /* not preclose_time_boolean */,
        transaction_closing_entry_exists(
            prior_date_string ) );

char *begin_date_string =
    statement_begin_date_string(
        prior_date_string );

LIST *prior_year_element_list =
    statement_fund_element_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        begin_date_string,
        transaction_date_time_nominal,
        transaction_date_time_fixed,
        fund_name,
        subclassification_option );

LIST *statement_prior_year_heading_list(
    LIST *prior_year_list );

LIST *statement_prior_year_delta_list(
    char *account_name,
    LIST *prior_year_list );

void statement_prior_year_list_net_income_set(
    LIST *prior_year_list,
    statement_fund_net_income() );

void statement_prior_year_net_income_set(
    STATEMENT_PRIOR_YEAR
        *statement_prior_year,
        statement_fund_net_income );

double revenue_total =
    element_seek(
        ELEMENT_REVENUE,
        prior_year_element_list )->
            element_current_balance() );

double expense_total =
    element_seek(
        ELEMENT_EXPENSE,
        prior_year_element_list )->
            element_current_balance() );

double gain_total =
    element_seek(
        ELEMENT_GAIN,
        prior_year_element_list )->
            element_current_balance() );

double loss_total =
    element_seek(
        ELEMENT_LOSS,
        prior_year_element_list )->
            element_current_balance() );

double statement_prior_year_net_income =
    statement_fund_net_income(
        revenue_total,
        expense_total,
        gain_total,
        loss_total );

int delta_prior =
    statement_delta_prior(
        statement_prior_year_net_income,
            statement_fund_net_income );
```

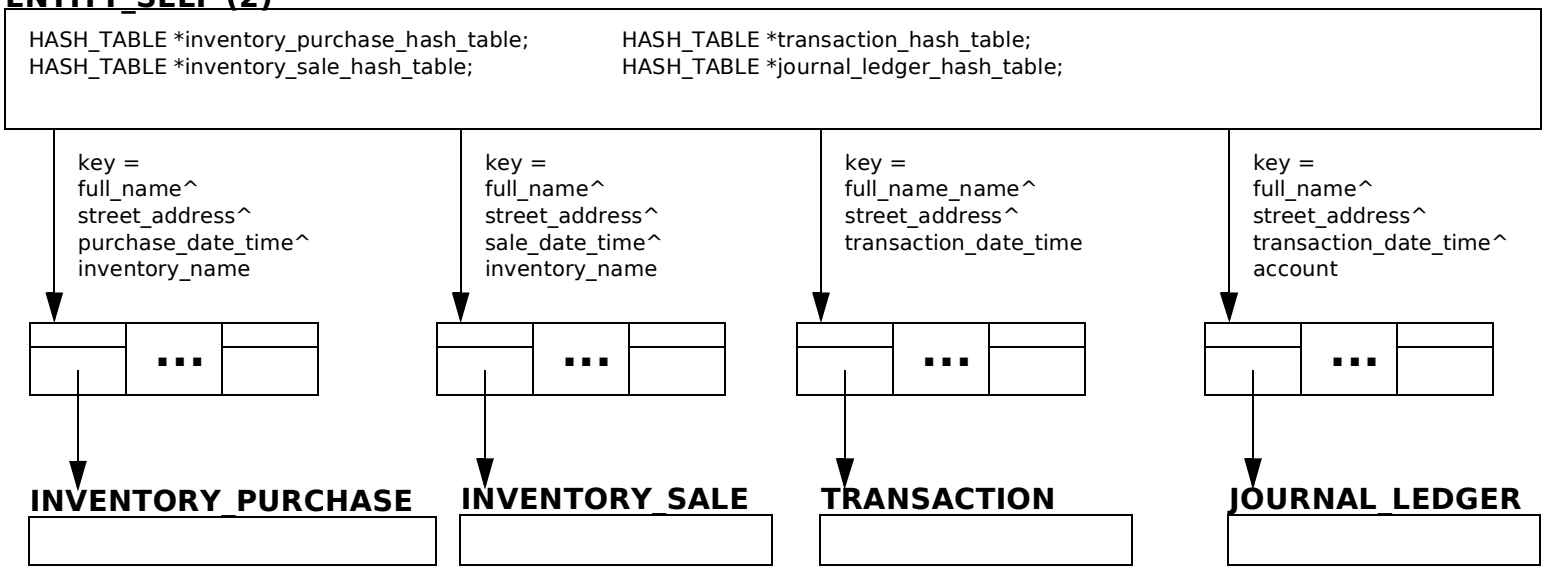


ENTITY

```
/* Usage */
ENTITY *entity_fetch(
      char *full_name,
      char *street_address );

/* Process */
char *entity_primary_where(
      char *full_name,
      char *street_address );

char *entity_system_string(
      char *ENTITY_SELECT,
      char *ENTITY_TABLE,
      char *entity_primary_where );

ENTITY *entity_parse(
      char *string_input() );

ENTITY *entity_new(
      char *strdup( full_name ),
      char *strdup( street_address ) );

ENTITY *entity_calloc( void );

/* Attributes */
char *full_name;
char *street_address;
char *unit;
char *city;
char *state_code;
char *zip_code;
char *phone_number;
char *email_address;

/* Public */
LIST *entity_full_street_list(
      LIST *full_name_list,
      LIST *street_address_list );
```

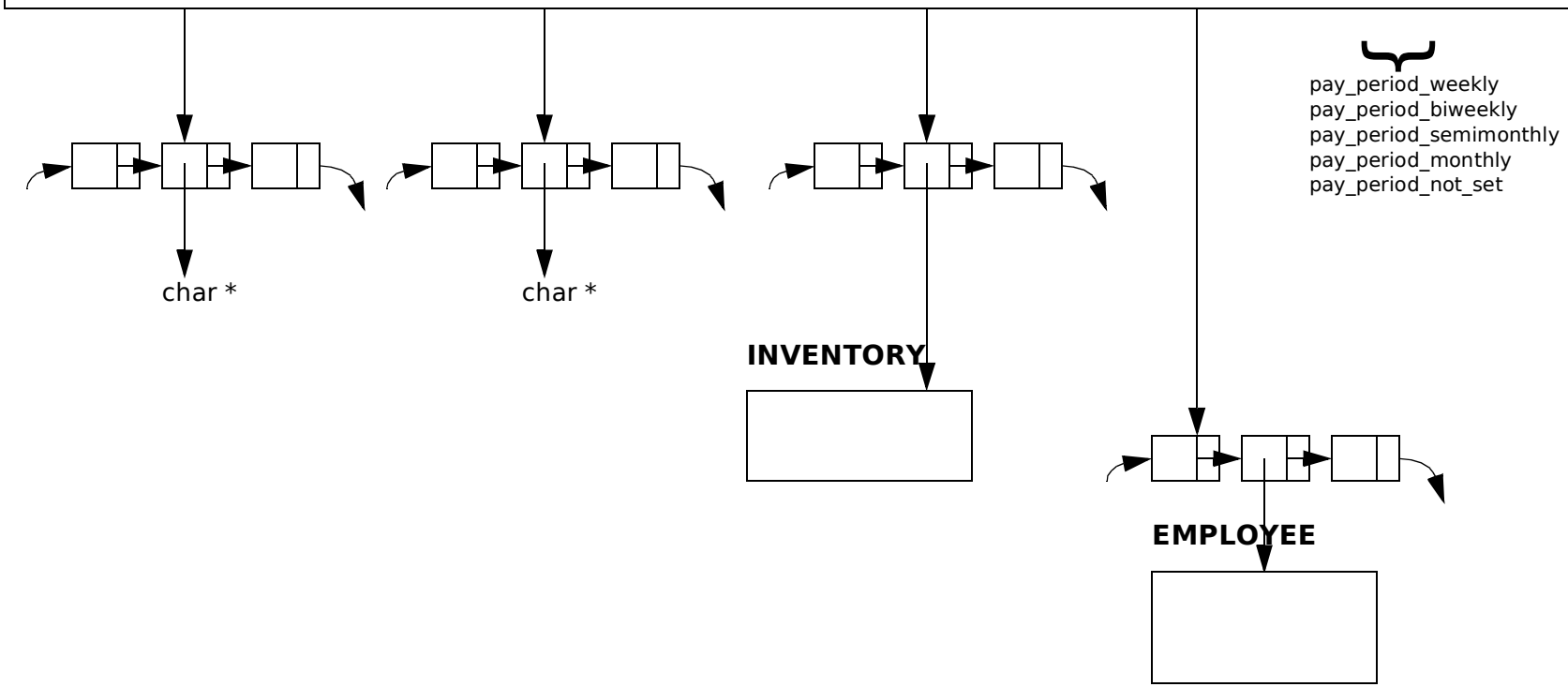

ENTITY_SELF (1)
ENTITY *entity;
LIST *inventory_purchase_list;
LIST *inventory_customer_sale_list;
INVENTORY *purchase_inventory;
INVENTORY *sale_inventory;
enum inventory_cost_method inventory_cost_method;
INVENTORY
inventory_not_set
inventory_fifo
inventory_average
inventory_fifo
PURCHASE
CUSTOMER_SALE
ENTITY
char *full_name;
char *street_address;
char *unit;
char *city;
char *state_code;
char *zip_code;
char *phone_number;
char *email_address;

ENTITY_SELF (2)



ENTITY_SELF (3)

```
LIST *inventory_purchase_name_list;
LIST *inventory_sale_name_list;
LIST *inventory_list;
LIST *employee_list;
enum payroll_pay_period payroll_pay_period;
```



ENTITY_SELF (4)

```
char *payroll_beginning_day
double social_security_combined_tax_rate
int social_security_payroll_ceiling
double medicare_combined_tax_rate
double medicare_additional_withholding_rate
int medicare_additional_gross_pay_floor
double federal_withholding_allowance_period_value
double federal_nonresident_withholding_income_premium
double state_withholding_allowance_period_value
double state_itemized_allowance_period_value
int federal_unemployment_wage_base
double federal_unemployment_tax_standard_rate
double federal_unemployment_threshold_rate
double federal_unemployment_tax_minimum_rate
int state_unemployment_wage_base
double state_unemployment_tax_rate
double state_sales_tax_rate;

char *entity_self_paypal_cash_account_name();
```

SALE (1)

```
ENTITY *customer_entity =
      entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *completed_date_time;
double shipping_revenue;
enum title_passage_rule title_passage_rule;

/* Returns sale_steady_state() */
SALE *sale_fetch(
      full_name,
      street_address,
      sale_date_time );

char *sale_primary_where(
      full_name,
      street_address,
      sale_date_time );

double sale_gross_revenue(
      inventory_sale_total(),
      fixed_service_sale_total(),
      hourly_service_sale_total() );

double sale_sales_tax(
      inventory_sale_list(),
      entity_self_sales_tax_rate() );

double sale_invoice_amount(
      sale_gross_revenue(),
      sale_sales_tax(),
      shipping_charge );

double Sale_amount_due(
      sale_invoice_amount(),
      customer_payment_total() )
{
      return A - B;
}

TRANSACTION *sale_transaction(
      full_name,
      street_address,
      completed_date_time,
      invoice_amount,
      gross_revenue,
      sale_tax,
      shipping_charge,
      account_receivable(),
      account_revenue(),
      account_shipping_revenue(),
      account_sales_tax_payable() );

void sale_update(
      inventory_sale_total(),
      fixed_service_sale_total(),
      hourly_service_sale_total(),
      sale_gross_revenue()
      sale_sales_tax(),
      sale_invoice_amount(),
      payment_total(),
      sale_amount_due(),
      sale_transaction()->
            transaction_date_time,
      full_name,
      street_address,
      sale_date_time );

SALE *sale_steady_state(
      full_name,
      street_address,
      sale_date_time,
      inventory_sale_list(),
      fixed_service_sale_list(),
      hourly_service_sale_list(),
      customer_payment_list(),
      shipping_charge,
      entity_self_sales_tax_rate(),
      completed_date_time,
      account_receivable(),
      account_revenue(),
      account_shipping_revenue(),
      account_sales_tax_payable() );
```

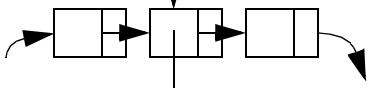
TRANSACTION

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

SALE (2)

```
LIST *customer_payment_list(
      full_name,
      street_address,
      sale_date_time );
```

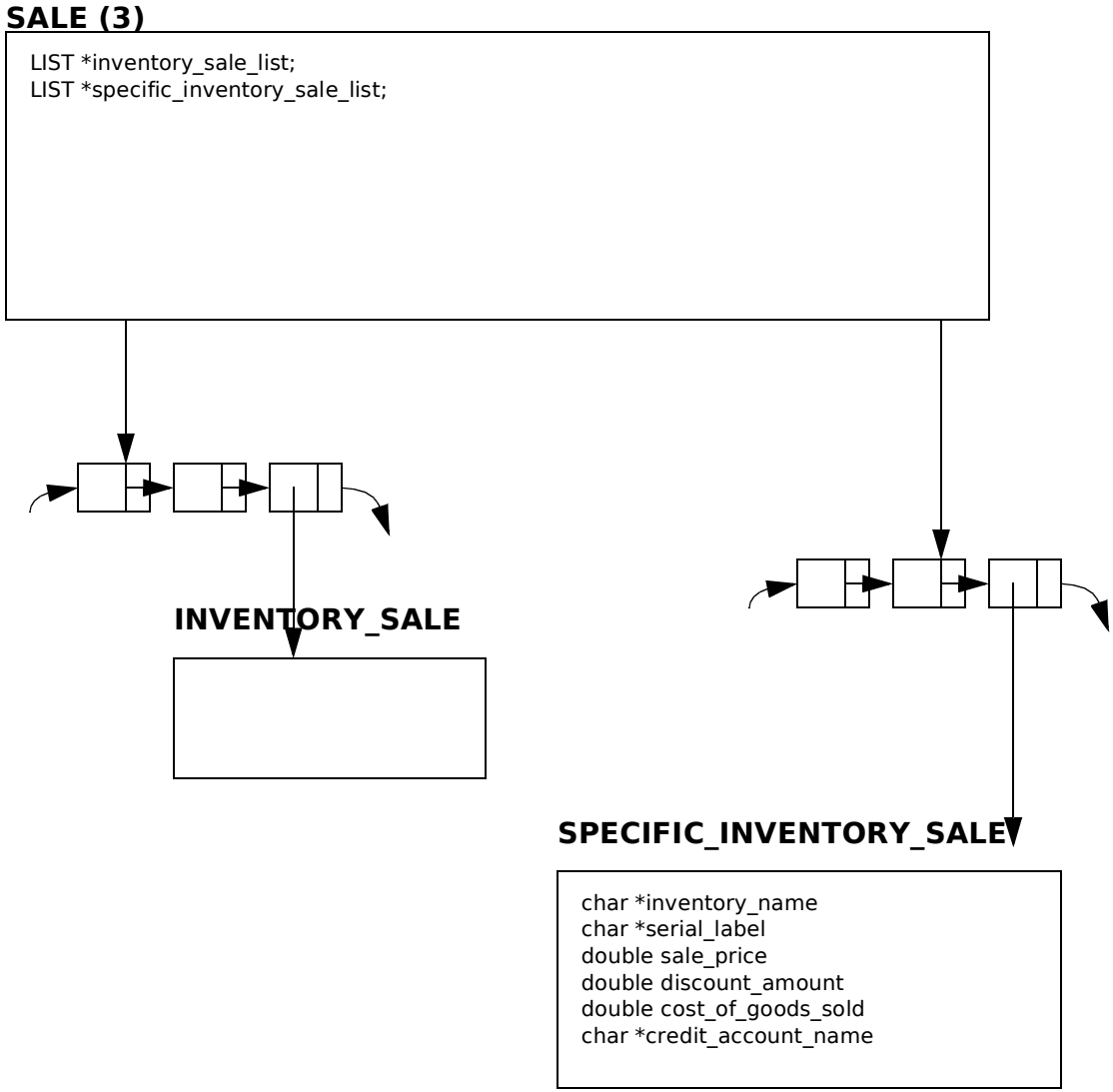


CUSTOMER_PAYMENT

```
ENTITY *customer_entity =
      entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

double customer_payment_total(
      customer_payment_list() )
{
      return sum( A->payment_amount );
}

CUSTOMER_PAYMENT *customer_payment_new(
      full_name,
      street_address,
      sale_date_time,
      payment_date_time );

char *customer_payment_select(
      void );

char *customer_payment_sys_string(
      where );

LIST *customer_payment_system_list(
      sys_string );

CUSTOMER_PAYMENT *customer_payment_parse(
      input );

TRANSACTION *customer_payment_transaction(
      full_name,
      street_address,
      payment_date_time,
      payment_amount,
      account_cash(),
      account_receivable() );
```

ENTITY

TRANSACTION


SALE (3)
LIST *inventory_sale_list;
LIST *specific_inventory_sale_list;
INVENTORY_SALE
SPECIFIC_INVENTORY_SALE
char *inventory_name
char *serial_label
double sale_price
double discount_amount
double cost_of_goods_sold
char *credit_account_name

HOURLY_SERVICE_SALE

```
ENTITY *customer_entity =
    entity_new(
        full_name,
        street_address );
char *sale_date_time;
char *service_name;
char *service_description;
double hourly_rate;
double discount_amount;
int estimated_hours;

char *hourly_service_sale_revenue_account_name(
    service_name,
    service_description );

LIST *hourly_service_work_list =
    work_hourly_list(
        full_name,
        street_address,
        sale_date_time,
        service_name,
        service_description );

double hourly_service_sale_work_hours =
    work_list_hours(
        hourly_service_work_list() );

double hourly_service_sale_net_revenue(
    hourly_rate,
    hourly_service_sale_work_hours,
    discount_amount )
{
    return (A x B) - C;
}
```

SALE (4)

```
LIST *    fixed_service_sale_list
LIST *    hourly_service_sale_list
```

FIXED_SERVICE_SALE

```
ENTITY *customer_entity =
    entity_new(
        full_name,
        street_address );
char *sale_date_time;
char *service_name;
double fixed_price;
double discount_amount;
int estimated_hours;
char *fixed_service_sale_revenue_accouint_name(
    service_name );

LIST *fixed_service_sale_work_list =
    work_fixed_list(
        full_name,
        street_address,
        sale_date_time,
        service_name );

double fixed_service_sale_work_hours =
    work_list_hours(
        fixed_service_sale_work_list() );

double fixed_service_sale_net_revenue(
    fixed_price,
    discount_amount )
{
    return A - B;
}
```

WORK

```
ENTITY *customer_entity = entity_new();
char *sale_date_time;
char *service_name;
char *service_description;
char *begin_date_time;
char *end_date_time;
char *login_name;
char *activity_name;

double work_hours(
    begin_date_time,
    end_date_time );

double work_list_hours(
    work_list );
```

PURCHASE (1)

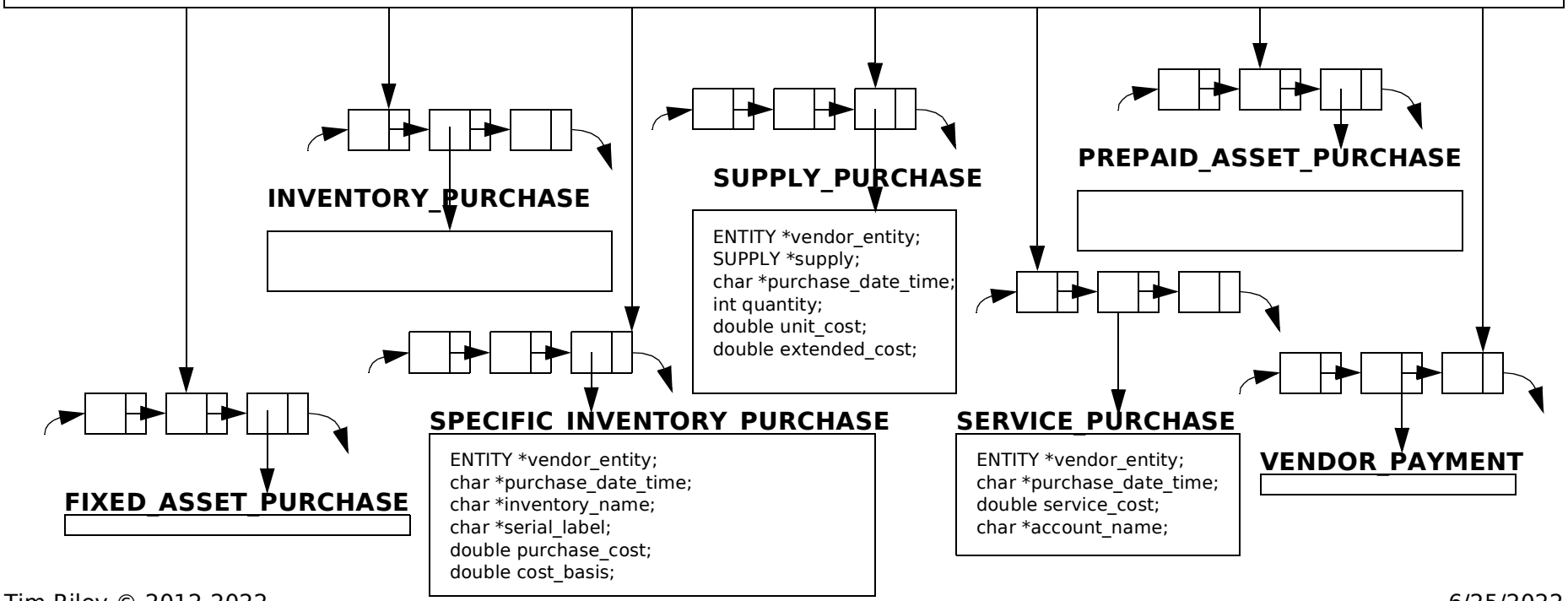
```
/* Usage */
PURCHASE *purchase_fetch(
    char *full_name,
    char *street_address,
    char *purchase_date_time );

/* Input */
ENTITY *vendor_entity =
    entity_new(
        full_name,
        street_address );

char *purchase_date_time;
double sales_tax;
double freight_in;
char *title_passage_rule_string;
char *shipped_date;
char *arrived_date_time;
char *program_name;
char *property_street_address;

/* Process */
enum title_passage_rule title_passage_rule =
    predictive_title_passage_rule_resolve(
        title_passage_rule_string );

char *purchase_primary_where(
    char *full_name,
    char *street_address,
    char *purchase_date_time );

char *purchase_system_string(
    char *where,
    char *order );

LIST *purchase_system_list(
    purchase_system_string() );

PURCHASE *purchase_parse(
    char *input );

LIST *fixed_asset_purchase_list =
    fixed_asset_purchase_list_fetch(
        vendor_entity->full_name,
        vendor_entity->street_address,
        purchase_date_time );

LIST *vendor_payment_list =
    vendor_payment_list_fetch(
        full_name,
        street_address,
        purchase_date_time );

PURCHASE *purchase_steady_state(
    sales_tax,
    freight_in,
    arrived_date,
    fixed_asset_purchase_list,
    inventory_purchase_list,
    specific_inventory_purchase_list,
    supply_purchase_list,
    prepaid_asset_purchase_list,
    inventory_purchase_return_list,
    vendor_payment_list,
    purchase );

double fixed_asset_purchase_total =
    fixed_asset_purchase_total(
        fixed_asset_purchase_list );

double inventory_purchase_total =
    inventory_purchase_total(
        inventory_purchase_list );

double specific_inventory_purchase_total =
    specific_inventory_purchase_total(
        specific_inventory_purchase_list );

double supply_purchase_total =
    supply_purchase_total(
        supply_purchase_list );

double prepaid_asset_purchase_total =
    prepaid_asset_purchase_total(
        prepaid_asset_purchase_list );

double inventory_purchase_return_total =
    inventory_purchase_return_total(
        inventory_purchase_return_list );

double vendor_payment_total =
    vendor_payment_total(
        vendor_payment_list );

double purchase_invoice_amount(
    fixed_asset_purchase_total,
    inventory_purchase_total,
    specific_inventory_purchase_total,
    supply_purchase_total,
    prepaid_asset_purchase_total,
    sales_tax,
    freight_in );

double purchase_amount_due(
        purchase_invoice_amount,
        vendor_payment_total )
{
    return A - B;
}

double purchase_asset_cost_basis(
        asset_cost,
        sales_tax,
        freight_in,
        fixed_asset_purchase_total(),
        inventory_purchase_total(),
        specific_inventory_purchase_total(),
        supply_expense_purchase_total(),
        prepaid_asset_purchase_total() );
```

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

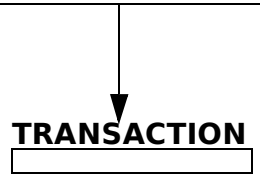
PURCHASE (2)

```
TRANSACTION *purchase_transaction =
    purchase_fixed_asset_transaction(
        full_name,
        street_address,
        arrived_date_time,
        purchase_invoice_amount,
        sales_tax,
        freight_in,
        fixed_asset_purchase_list(),
        account_payable() );

/* Returns true transaction_date_time */
char *purchase_transaction_refresh(
    purchase_transaction->
        transaction_amount,
    purchase_transaction->
        journal_list,
    purchase_transaction->memo,
    full_name,
    street_address,
    purchase_transaction->
        transaction_date_time );

void purchase_update(
    fixed_asset_purchase_total,
    purchase_invoice_amount,
    vendor_payment_total,
    purchase_amount_due,
    purchase_transaction->
        transaction_date_time,
    full_name,
    street_address,
    purchase_date_time );

LIST *inventory_purchase_list
LIST *specific_inventory_purchase_list
LIST *service_purchase_list
LIST *supply_purchase_list
LIST *prepaid_asset_purchase_list;
char *property_street_address;
```



VENDOR_PAYMENT

```
/* Usage */
VENDOR_PAYMENT *vendor_payment_fetch(
	char *full_name,
	char *street_address,
	char *purchase_date_time,
	char *payment_date_time );

LIST *vendor_payment_list_fetch(
	full_name,
	street_address,
	purchase_date_time );

/* Input */
ENTITY *vendor_entity;
char *purchase_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

/* Process */
VENDOR_PAYMENT *vendor_payment_new(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time );

char *vendor_payment_primary_where(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time );

char *vendor_payment_system_string(
	purchase_primary_where() );

LIST *vendor_payment_system_list(
	vendor_payment_system_string() );

VENDOR_PAYMENT *vendor_payment_parse(
	input );

double vendor_payment_total(
	LIST *vendor_payment_system_list() )
{
	return sum(
			A->
			vendor_payment->
			payment_amount );
}

TRANSACTION *vendor_payment_transaction(
	full_name,
	street_address,
	payment_date_time,
	payment_amount,
	check_number,
	account_payable(),
	account_cash(),
	account_uncleared_checks() );

LIST *vendor_payment_journal_list(
	payment_amount,
	check_number,
	account_payable(),
	account_cash(),
	account_uncleared_checks() );

void vendor_payment_insert(
	full_name,
	street_address,
	purchase_date_time,
	payment_date_time,
	payment_amount,
	check_number );

VENDOR_PAYMENT *vendor_payment_seek(
	vendor_payment_list,
	payment_date_time );

char *vendor_payment_update_system_string(
	void );
```



FIXED_ASSET_PURCHASE

```
/* Usage */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_fetch(
        char *asset_name,
        char *serial_label,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

LIST *fixed_asset_purchase_list_fetch(
        char *where,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

/* Input */
FIXED_ASSET *fixed_asset;
char *serial_label;
ENTITY *vendor_entity;
char *purchase_date_time;
char *service_placement_date;
double fixed_asset_cost;
int units_produced_so_far;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
enum depreciation_method depreciation_method =
        depreciation_method_resolve();
int estimated_useful_life_years;
int estimated_useful_life_units;
int estimated_residual_value;
int declining_balance_n;
double finance_accumulated_depreciation;
double tax_adjusted_basis;

/* Process */
FIXED_ASSET_PURCHASE *fixed_asset_purchase_new(
        char *asset_name,
        char *serial_number );

FIXED_ASSET *fixed_asset =
        fixed_asset_fetch(
            asset_name );

char *fixed_asset_purchase_primary_where(
        char *asset_name,
        char *serial_label );

char *fixed_asset_purchase_system_string(
        char *where,
        char *order );

LIST *fixed_asset_purchase_system_list(
        fixed_asset_purchase_system_string(),
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

FIXED_ASSET_PURCHASE *fixed_asset_purchase_parse(
        char *input,
        boolean fetch_last_depreciation,
        boolean fetch_last_recovery );

double cost_basis =
        purchase_cost_basis(
            fixed_asset_cost,
            purchase->sales_tax,
            purchase->freight_in,
            fixed_asset_purchase_total(),
            inventory_purchase_total(),
            specific_inventory_purchase_total(),
            supply_expense_purchase_total(),
            prepaid_asset_purchase_total() );

double finance_accumulated_depreciation =
        depreciation_accumulated_depreciation();

double tax_accumulated_depreciation =
        recovery_accumulated_depreciation();

LIST *fixed_asset_purchase_list_depreciate(
        fixed_asset_purchase_list_fetch(
            "finance_accumulated_depreciation <
             cost_basis",
            0 /* not fetch_last_depreciation */,
            0 /* not fetch_last_recovery */ ),
        date_yyyy_mm_dd_now()
            /* depreciation_date */ );

DEPRECIATION *depreciation =
        depreciation_evaluate(
            fixed_asset->asset_name,
            fixed_asset_purchase->serial_label,
            service_placement_date,
            depreciation_prior_depreciation_date(),
            date_yyyy_mm_dd_now()
                /* depreciation_date */,
            fixed_asset_purchase_cost_basis(),
            units_produced_so_far,
            estimated_residual_value,
            estimated_useful_life_years,
            estimated_useful_life_units,
            declining_balance_n,
            finance_accumulated_depreciation
                /* prior_accumulated_depreciation */,
            enum depreciation_method ) );

LIST *fixed_asset_purchase_depreciation_list(
        LIST *fixed_asset_purchase_list );

void fixed_asset_purchase_list_update(
        LIST *fixed_asset_purchase_list );

FILE *fixed_asset_purchase_update_open(
        void );

void fixed_asset_purchase_update(
        update_pipe,
        fixed_asset_purchase_cost_basis(),
        finance_accumulated_depreciation,
        tax_adjusted_basis,
        asset_name,
        serial_label );

void fixed_asset_purchase_finance_fetch_update(
        char *asset_name,
        char *serial_label );

RECOVERY *recovery =
        recovery_evaluate();

void fixed_asset_purchase_cost_fetch_update(
        char *asset_name,
        char *serial_label );

LIST *fixed_asset_purchase_list_cost_recover(
        LIST *fixed_asset_purchase_list,
        int tax_year );

LIST *fixed_asset_purchase_cost_recovery_list(
        LIST *fixed_asset_purchase_list );
```

ENTITY **FIXED_ASSET** **DEPRECIATION** **RECOVERY**

FIXED_ASSET

```
/* Usage */
FIXED_ASSET *fixed_asset_fetch(
      char *asset_name );

/* Attributes */
char *asset_name;
char *account_name;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;
double activity_energy_kilowatt_draw;
double activity_depreciation_per_hour;

/* Process */
FIXED_ASSET *fixed_asset_new(
      char *asset_name );

FIXED_ASSET *fixed_asset_parse(
      char *input );

char *fixed_asset_primary_where(
      char *asset_name );

char *fixed_asset_system_string(
      char *where );

char *fixed_asset_name_escape(
      char *fixed_asset );
```

RECOVERY

```
/* Usage */
RECOVERY *recovery_evaluate(
    char *asset_name,
    char *serial_label,
    int tax_year,
    double fixed_asset_purchase->
        cost_basis,
    char *fixed_asset_purchase->
        service_placement_date,
    char *fixed_asset_purchase->
        disposal_date,
    char *fixed_asset_purchase->
        cost_recovery_period_string,
    char *fixed_asset_purchase->
        cost_recovery_method,
    char *fixed_asset_purchase->
        cost_recovery_conversion );

RECOVERY *recovery_fetch(
    char *asset_name,
    char *serial_label,
    int tax_year );

LIST *recovery_list_fetch(
    char *asset_name,
    char *serial_label );

void recovery_fixed_asset_list_set(
    LIST *fixed_asset_list,
    int tax_year );

/* Input */
char *asset_name;
char *serial_label;
int tax_year;
double cost_basis;
char *service_placement_date;
char *disposal_date;
char *cost_recovery_period_string;
char *cost_recovery_method;
char *cost_recovery_conversion;

/* Process */
char *recovery_system_string(
    char *where );

LIST *recovery_system_list(
    recovery_system_string() );

char *recovery_primary_where(
    char *asset_name,
    char *serial_label,
    int tax_year );

double recovery_period_years(
    char *cost_recovery_period_string );

int service_placement_month =
    date_month(
        date_yyyy_mm_dd_new(
            service_placement_date ) );

int service_placement_year =
    date_year(
        date_yyyy_mm_dd_new(
            service_placement_date ) );

int disposal_month =
    date_month(
        date_yyyy_mm_dd_new(
            disposal_date ) );

int disposal_year =
    date_year(
        date_yyyy_mm_dd_new(
            disposal_date ) );

int recovery_service_placement_years(
    int tax_year,
    int service_placement_year )
{
    return A - B + 1;
}

char *recovery_subquery_where(
    int tax_year );

RECOVERY_STRAIGHT_LINE *straight_line =
    recovery_statutory_straight_line_evaluate(
        tax_year,
        cost_basis,
        service_placement_month,
        service_placement_year,
        disposal_month,
        disposal_year,
        recovery_period_years(),
        cost_recovery_conversion );

RECOVERY_ACCELERATED *accelerated =
    recovery_statutory_accelerated_evaluate(
        tax_year,
        cost_basis,
        service_placement_month,
        service_placement_year,
        disposal_month,
        disposal_year,
        recovery_period_years(),
        cost_recovery_conversion );

void recovery_fixed_asset_list_display(
    FILE *output_pipe,
    LIST *fixed_asset_list );

void recovery_list_insert(
    LIST *recovery_list );

double recovery_rate(
    straight_line,
    accelerated );

double recovery_amount(
    straight_line,
    accelerated );
```

RECOVERY_STRAIGHT_LINE **RECOVERY_ACCELERATED**

RECOVERY_STRAIGHT_LINE

```
/* Usage */
RECOVERY_STRAIGHT_LINE *
    recovery_statutory_straight_line_evaluate(
        int tax_year,
        double cost_basis,
        int service_placement_month,
        int service_placement_year,
        int disposal_month,
        int disposal_year,
        double recovery_period_years,
        char *cost_recovery_conversion );

/* Process */
RECOVERY_STRAIGHT_LINE *recovery_straight_line_calloc();

double service_placement_years =
    recovery_service_placement_years(
        tax_year,
        service_placement_year );

double straight_line_rate =
    recovery_statutory_straight_line_half_year_rate(
        int recovery_service_placement_years(),
        int tax_year,
        int disposal_year,
        double recovery_period_years );

double recovery_statutory_straight_line_half_year_mid_year_rate(
    double recovery_period_years );

double recovery_statutory_straight_line_half_year_book_ends_rate(
    double recovery_statutory_straight_line_mid_year_rate() );

double straight_line_amount =
    recovery_straight_line_amount(
        double straight_line_rate,
        double cost_basis )
{
    return A X B;
}
```

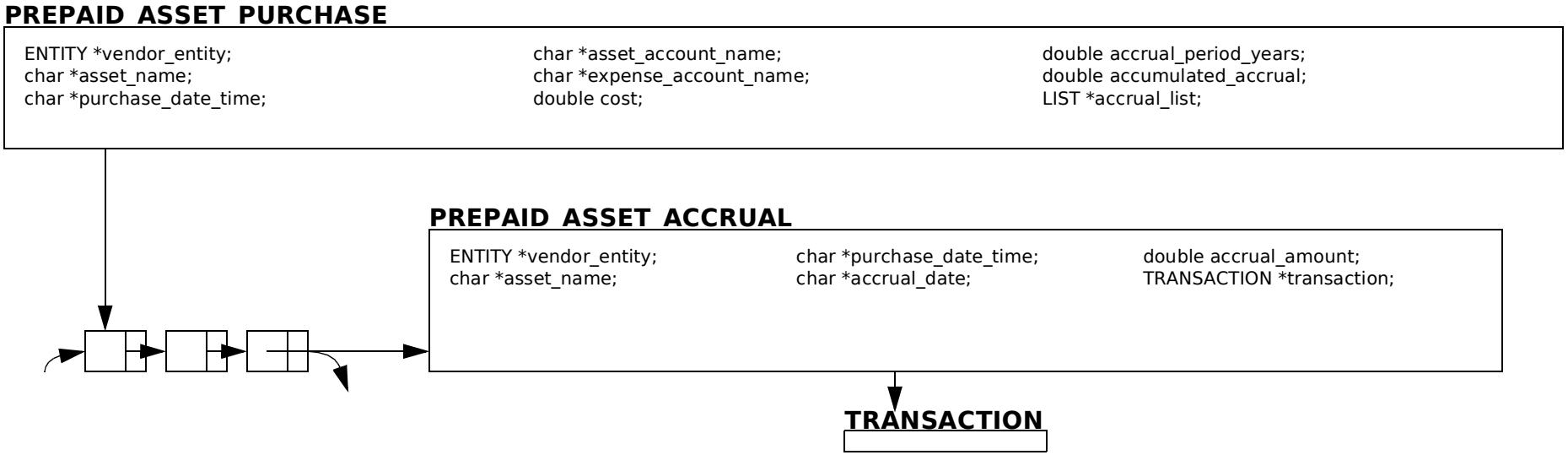
RECOVERY_ACCELERATED

```
/* Usage */
RECOVERY_ACCELERATED *
    recovery_statutory_accelerated_evaluate(
        int tax_year,
        double cost_basis,
        int service_placement_month,
        int service_placement_year,
        int disposal_month,
        int disposal_year,
        double recovery_period_years,
        char *cost_recovery_conversion );

/* Process */
RECOVERY_ACCELERATED *recovery_accelerated_calloc();

double accelerated_rate =
    recovery_statutory_accelerated_half_year_rate(
        int recovery_service_placement_years(),
        int tax_year,
        int disposal_year,
        double recovery_period_years );

double recovery_statutory_accelerated_half_year_evaluate(
    int service_placement_years,
    double recovery_period_years );

double accelerated_amount =
    recovery_accelerated_amount(
        double accelerated_rate,
        double cost_basis );
```

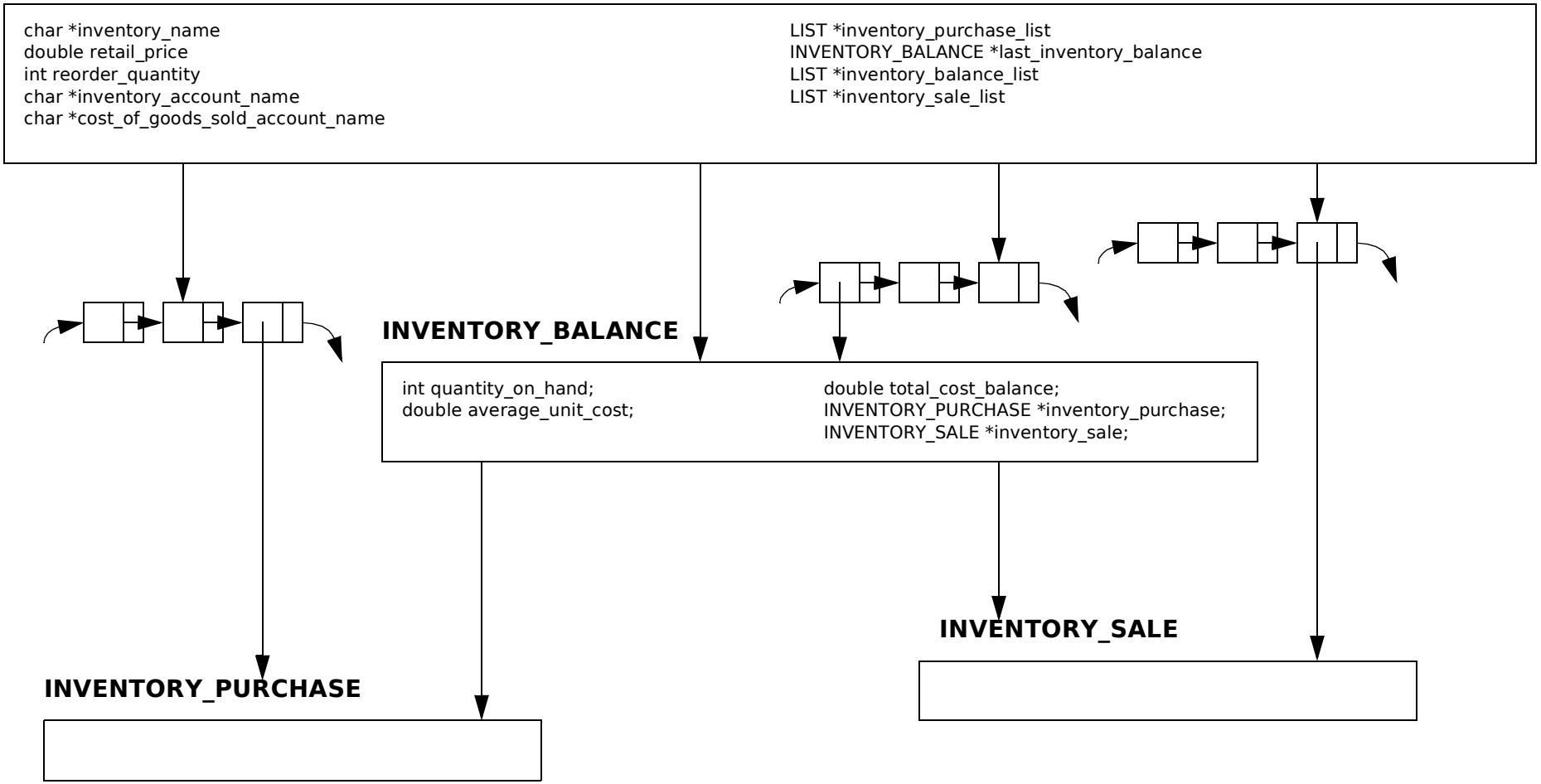
DEPRECIATION

```
/* Usage */
DEPRECIATION *depreciation_evaluate(
        char *asset_name,
        char *serial_label,
        depreciation_method_evaluate(),
        char *service_placement_date,
        char *depreciation_prior_depreciation_date(),
        char *date_yyyy_mm_dd_now()
                /* depreciation_date */,
        double cost_basis,
        int units_produced_so_far,
        int estimated_residual_value,
        int estimated_useful_life_years,
        int estimated_useful_life_units,
        int declining_balance_n,
        double fixed_asset_purchase->
                finance_accumulated_depreciation
                  /* prior_accumulated_depreciation */ );

DEPRECIATION *depreciation_fetch(
        char *asset_name,
        char *serial_label,
        char *depreciation_date );

/* Process */
enum depreciation_method =
        depreciation_method_evaluate(
                depreciation_method_string );

char *depreciation_prior_depreciation_date(
        char *asset_name,
        char *serial_label,
        char *current_depreciation_date );

double depreciation_amount(
        enum depreciation_method,
        char *service_placement_date,
        depreciation_prior_depreciation_date(),
        char *depreciation_date,
        double cost_basis,
        depreciation_units_produced_current(),
        int estimated_residual_value,
        int estimated_useful_life_years,
        int estimated_useful_life_units,
        int declining_balance_n,
        double prior_accumulated_depreciation );

double depreciation_straight_line(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        double prior_accumulated_depreciation );

double depreciation_sum_of_years_digits(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        prior_accumulated_depreciation );

double depreciation_double_declining_balance(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        double prior_accumulated_depreciation );

double depreciation_n_declining_balance(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date,
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_years,
        int declining_balance_n,
        double prior_accumulated_depreciation );

int depreciation_units_produced_total(
        char *asset_name,
        char *serial_label );

int depreciation_units_produced_current(
        int units_produced_so_far,
        depreciation_units_produced_total() );

double depreciation_units_of_production(
        double cost_basis,
        double estimated_residual_value,
        int estimated_useful_life_units,
        int depreciation_units_produced_current(),
        double prior_accumulated_depreciation );

double depreciation_fraction_of_year(
        char *service_placement_date,
        char *prior_depreciation_date,
        char *depreciation_date_string );

double depreciation_accumulated_depreciation(
        prior_accumulated_depreciation,
        depreciation_amount() );

char *depreciation_subquery_where(
        char *depreciation_date );

TRANSACTION *depreciation_transaction(
        entity_self->entity->full_name,
        entity_self->entity->street_address,
        predictive_transaction_date_time(
                depreciation_date ),
        depreciation_amount(),
        account_depreciation_expense(),
        account_accumulated_depreciation() );

void depreciation_insert(
        FILE *insert_pipe,
        char *asset_name,
        char *serial_label,
        char *depreciation_date,
        int depreciation_units_produced_current(),
        double depreciation_amount,
        char *depreciation_transaction()->
                transaction_date_time );

void depreciation_list_negate_depreciation_amount(
        LIST *depreciation_list );

LIST *depreciation_transaction_list(
        fixed_asset_purchase_depreciation_list() );
```

straight_line,
double_declining_balance,
n_declining_balance,
sum_of_years_digits,
units_of_production,
not_depreciated


PREPAID_ASSET_PURCHASE
ENTITY *vendor_entity;
char *asset_name;
char *purchase_date_time;
char *asset_account_name;
char *expense_account_name;
double cost;
double accrual_period_years;
double accumulated_accrual;
LIST *accrual_list;
PREPAID_ASSET_ACCRUAL
ENTITY *vendor_entity;
char *asset_name;
char *purchase_date_time;
char *accrual_date;
double accrual_amount;
TRANSACTION *transaction;
TRANSACTION

INVENTORY


char *inventory_name
double retail_price
int reorder_quantity
char *inventory_account_name
char *cost_of_goods_sold_account_name
LIST *inventory_purchase_list
INVENTORY_BALANCE *last_inventory_balance
LIST *inventory_balance_list
LIST *inventory_sale_list
INVENTORY_BALANCE
int quantity_on_hand;
double average_unit_cost;
double total_cost_balance;
INVENTORY_PURCHASE *inventory_purchase;
INVENTORY_SALE *inventory_sale;
INVENTORY_SALE
INVENTORY_PURCHASE

INVENTORY_PURCHASE

char *	full_name;	int	layer_consumed_quantity;
char *	street_address;	double	average_unit_cost;
char *	purchase_date_time;	char *	arrived_date_time;
char *	inventory_name;	char *	inventory_account_name;
int	ordered_quantity;	LIST *	inventory_purchase_return_list;
double	unit_cost;		
double	capitalized_unit_cost;		
double	extended_cost;		
int	arrived_quantity;		
int	missing_quantity;		
int	quantity_on_hand;		

INVENTORY_PURCHASE_RETURN

char *	return_date_time
int	returned_quantity
double	sales_tax
char *	transaction_date_time
TRANSACTION *	transaction

TRANSACTION

INVENTORY_SALE

char *	full_name;	double	extended_price;
char *	street_address;	double	cost_of_goods_sold;
char *	sale_date_time;	char *	completed_date_time;
char *	inventory_name;	char *	inventory_account_name;
int	sold_quantity;	char *	cost_of_goods_sold_account_name;
double	retail_price;	LIST *	inventory_sale_return_list;
double	discount_amount;	LIST *	layer_inventory_purchase_list;

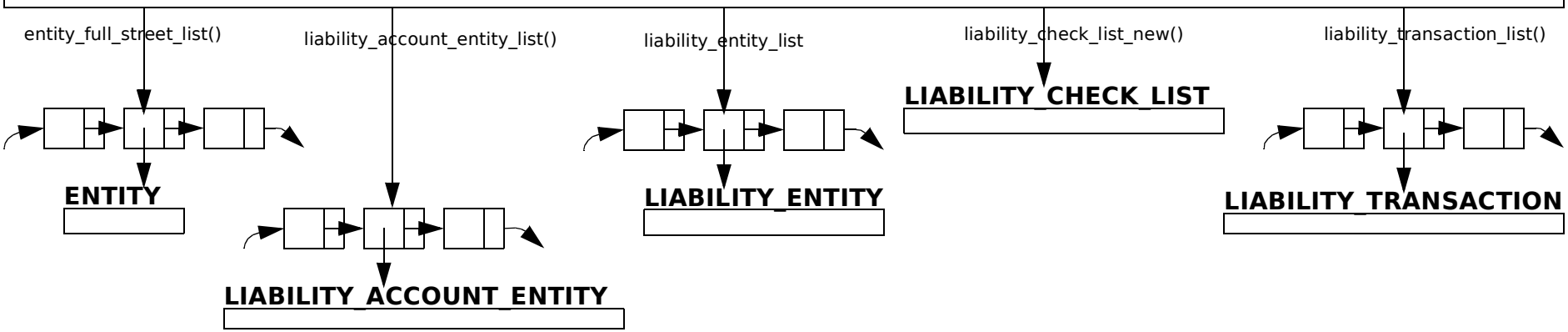
INVENTORY_SALE_RETURN

char *	return_date_time;
int	returned_quantity;
char *	transaction_date_time;
TRANSACTION *	transaction;

TRANSACTION

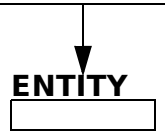
LIABILITY_PAYMENT

```
/* Usage */
LIABILITY_PAYMENT *liability_payment_new(
   char *application_name,
   double dialog_box_payment_amount,
   int starting_check_number,
   char *appaserver_data_directory,
   char *document_root_directory,
   char *process_name,
   char *session_key,
   LIST *entity_full_street_list(
         full_name_list,
         street_address_list ) );

/* Process */
if ( !list_length( entity_full_street_list() )
{
   return (LIABILITY_PAYMENT *)0;
}

LIABILITY_PAYMENT *liability_payment_calloc( void );

LIST *liability_account_entity_list();

LIST *account_current_liability_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_CURRENT_LIABILITY,
   ACCOUNT_UNCLEARED_CHECKS );

LIST *account_receivable_name_list(
   ACCOUNT_TABLE,
   SUBCLASSIFICATION_RECEIVABLE );

LIST *liability_entity_list = list_new();

if ( list_length( entity_full_street_list ) == 1 )
{
   list_set(
         liability_entity_list,
         liability_entity_new(
               dialog_box_payment_amount,
               liability_account_entity_list(),
               account_current_liability_name_list(),
               account_receivable_name_list(),
               list_first( entity_full_street_list ) ) );
}
else
{
   for ENTITY *entity in entity_full_street_list
   {
         list_set(
               liability_entity_list,
               liability_entity_new(
                     0 /* dialog_box_payment_amount */,
                     liability_account_entity_list(),
                     account_current_liability_name_list(),
                     account_receivable_name_list(),
                     entity ) );
   }
}

if ( !list_length( liability_entity_list ) )
{
   return (LIABILITY_PAYMENT *)0;
}

if ( starting_check_number )
{
   LIABILITY_CHECK_LIST *liability_check_list -=
         liability_check_list_new(
               application_name,
               starting_check_number,
               appaserver_data_directory,
               document_root_directory,
               process_name,
               session_key,
               liability_entity_list );
}

LIST *liability_transaction_list(
   starting_check_number,
   liability_entity_list );

/* Driver */
if ( liability_check_list )
{
      appaserver_library_output_ftp_prompt(
            liability_check_list->
                  liability_check_appaserver_link->
                  ftp_filename,
            LIABILITY_PROMPT,
            "check_page" /* target_frame */,
            (char *)0 /* application_type */ );
}

if ( execute )
{
      void liability_transaction_list_insert(
            liability_transaction_list() );
}
```



LIABILITY_ACCOUNT_ENTITY

```
/* Usage */
LIST *liability_account_entity_list( void );

/* Process */
char *liability_account_entity_system_string(
        char *LIABILITY_ACCOUNT_ENTITY_SELECT,
        char *LIABILITY_ACCOUNT_ENTITY_TABLE );

LIST *liability_account_entity_system_list(
        char *liability_account_entity_system_string() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_parse(
                char *string_input() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_new(
                char *strdup( account_name ) );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_calloc(
                void );

ENTITY *entity =
        entity_new(
                strdup( full_name ),
                strdup( street_address ) );

/* Attributes */
char *account_name;
ENTITY *entity;

/* Public */
LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_seek_account(
                char *account_name,
                LIST *liability_account_entity_list() );

LIABILITY_ACCOUNT_ENTITY *
        liability_account_entity_seek_entity(
                char *full_name,
                char *street_address,
                LIST *liability_account_entity_list() );
```



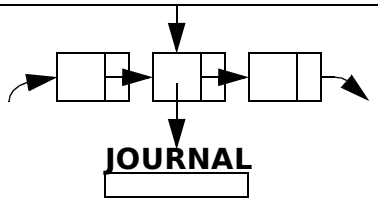
LIABILITY_ENTITY

```
/* Usage */
LIABILITY_ENTITY *liability_entity_new(
   double dialog_box_payment_amount,
   LIST *liability_account_entity_list(),
   LIST *account_current_liability_name_list(),
   LIST *account_receivable_name_list(),
   ENTITY *entity );

/* Process */
LIABILITY_ENTITY *liability_entity_calloc( void );

if ( !entity ) exit( 1 );

LIABILITY_ACCOUNT_ENTITY *
      liability_account_entity =
            liability_account_entity_seek_entity(
            entity->full_name,
            entity->street_address,
            liability_account_entity_list );

if ( liability_account_entity )
{
      LIABILITY *liability =
            liability_account_fetch(
                  liability_account_entity->
                        accout_name );
}
else
{
      LIABILITY *liability =
            liability_entity_fetch(
            entity->full_name,
            entity->street_address,
            account_current_liability_name_list );
}

if ( !liability )
{
   return (LIABILITY_ENTITY *)0;
}

if ( !liability_account_entity )
{
      RECEIVABLE *receivable =
            receivable_fetch(
                  entity->full_name,
                  entity->street_address,
                  account_receivable_name_list );
}

double liability_entity_amount_due(
      double dialog_box_payment_amount,
      LIABILITY *liability,
      RECEIVABLE *receivable );
```

LIABILITY_ACCOUNT_ENTITY **LIABILITY** **RECEIVABLE**

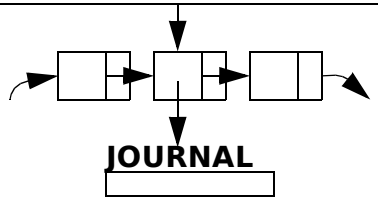
LIABILITY

```
/* Usage */
LIABILITY *liability_entity_fetch(
    char *full_name,
    char *street_address,
    LIST *account_current_liability_name_list() );

/* Process */
LIABILITY *liability_calloc();

char *timlib_in_clause(
    account_current_liability_name_list );

char *liability_entity_where(
    char *full_name,
    char *street_address,
    char *timlib_in_clause() );

LIST *journal_system_list(
    journal_system_string(
        JOURNAL_SELECT,
        JOURNAL_TABLE,
        liability_entity_where() ),
    0 /* not fetch_check_number */,
    0 /* not fetch_memo */ );

double journal_credit_debit_difference_sum(
    journal_system_list() );

/* Usage */
LIABILITY *liability_account_fetch(
    char *liability_accout_name );

/* Process */
LIABILITY *liability_calloc();

char *liability_account_where(
    char *liability_account_name );

LIST *journal_system_list(
    journal_system_string(
        JOURNAL_SELECT,
        JOURNAL_TABLE,
        liability_account_where() ),
    0 /* not fetch_check_number */,
    0 /* not fetch_memo */ );

double journal_credit_debit_difference_sum(
    journal_system_list() );

/* Private */
LIABILITY *liability_calloc( void );
```



RECEIVABLE

```
/* Usage */
RECEIVABLE *receivable_fetch(
    char *full_name,
    char *street_address,
    LIST *account_receivable_name_list() );

/* Process */
RECEIVABLE *receivable_calloc( void );

char *timlib_in_clause(
    account_receivable_name_list );

char *receivable_where(
    char *full_name,
    char *street_address,
    char *timlib_in_clause() );

LIST *journal_system_list(
    journal_system_string(
        JOURNAL_SELECT,
        JOURNAL_TABLE,
        receivable_where() ),
    0 /* not fetch_check_number */,
    0 /* not fetch_memo */ );

double journal_debit_credit_difference_sum(
    journal_system_list() );
```



LIABILITY_CHECK_LIST

```
/* Usage */
LIABILITY_CHECK_LIST *liability_check_list_new(
    char *application_name,
    char *appaserver_data_directory,
    char *document_root_directory,
    char *process_name,
    char *session_key,
    LIST *liability_entity_list() );

/* Process */
LIABILITY_CHECK_LIST *liability_check_list_calloc( void );

LIST *check_list = list_new();

char *liability_check_list_documentclass( void );

char *liability_check_list_usepackage( void );

char *liability_check_list_pagenumbering_gobble( void );

char *liability_check_list_begin_document( void );

char *liability_check_list_heading(
    char *liability_check_list_documentclass(),
    char *liability_check_list_usepackage(),
    char *liability_check_list_pagenumbering_gobble(),
    char *liability_check_list_begin_document() );

for LIABILITY_ENTITY *liability_entity in
    liability_enitity_list
{
    list_set(
        check_list,
        liability_check_new(
            check_personal_size,
            starting_check_number++
                /* check_number */,
            liability_entity ) );
}

char *liability_check_list_end_document( void );

LIABILITY_CHECK_APPASERVER_LINK *
    liability_check_appaserver_link =
        liability_check_appaserver_link_new(
            application_name,
            document_root_directory,
            process_name,
            session_key );

/* Driver */
FILE *output_file =
    fopen(
        liability_check_appaserver_link->
            output_filename,
        "w" );

fprintf(  output_file,
          "%s\n",
          liability_check_list_heading() );

for LIABILITY_CHECK *liability_check in check_list
{
    fprintf(  output_file,
              "%s\n",
              liability_check->output_string );
}

fprintf(  output_file,
          "%s\n",
          liability_check_list_end_document() );

fclose( output_file );

latex_tex2pdf(
    liability_check_appaserver_link->
        output_filename,
    appaserver_data_directory
        /* working_directory */ );

sprintf(  system_string,
          "cp %s %s",
          liability_check_appaserver_link->
              pdf_filename,
          liability_check_appaserver_link->
              document_root_filename );

system( system_string );
```

LIABILITY_CHECK_APPASERVER_LINK

```
/* Usage */
LIABILITY_CHECK_APPASERVER_LINK *
    liability_check_appaserver_link_new(
    char *application_name,
    char *document_root_directory,
    char *process_name,
    char *session_key );

/* Process */
LIABILITY_CHECK_APPASERVER_LINK *
    liability_check_appaserver_link_calloc(
        void );

APPASERVER_LINK_FILE *appaserver_link_file =
    appaserver_link_file_new(
        application_http_prefix( application_name ),
        appaserver_library_get_server_address(),
        ( application_prepend_http_protocol_yn(
            application_name ) == 'y' ),
        document_root_directory,
        process_name /* filename_stem */,
        application_name,
        getpid(),
        (char *)0 /* session_key */,
         "tex" );

char *output_filename =
    appaserver_link_get_output_filename(
        appaserver_link_file->
            output_file->
            document_root_directory,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

appaserver_link_file->extension = "pdf";

char *pdf_output_filename =
    appaserver_link_get_link_prompt(
        appaserver_link_file->
            link_prompt->
            prepend_http_boolean,
        appaserver_link_file->
            link_prompt->
            http_prefix,
        appaserver_link_file->
            link_prompt->
            server_address,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

char *ftp_filename =
    appaserver_link_get_link_prompt(
        appaserver_link_file->
            link_prompt->
            prepend_http_boolean,
        appaserver_link_file->
            link_prompt->
            http_prefix,
        appaserver_link_file->
            link_prompt->
            server_address,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );

appaserver_link_file->extension = "pdf";
appaserver_link_file->session = session_key;
appaserver_link_file->process_id = 0;

char *document_root_filename =
    appaserver_link_get_output_filename(
        appaserver_link_file->
            output_file->
            document_root_directory,
        appaserver_link_file->application_name,
        appaserver_link_file->filename_stem,
        appaserver_link_file->begin_date_string,
        appaserver_link_file->end_date_string,
        appaserver_link_file->process_id,
        appaserver_link_file->session,
        appaserver_link_file->extension );
```

APPASERVER_LINK_FILE

LIABILITY_CHECK

```
/* Usage */
LIABILITY_CHECK *liability_check_new(
	int check_number,
	LIABILITY_ENTITY *liability_entity );

/* Process */
LIABILITY_CHECK *liability_check_calloc( void );

char *liability_check_dollar_text(
	double liability_entity->amount_due );

char *liability_check_payable_to(
	char *liability_entity->entity->full_name );
```

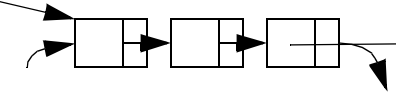
EMPLOYEE

char *full_name
char *street_address
double hourly_wage
double period_salary
double commission_sum_percent
double gross_pay_year_to_date
double net_pay_year_to_date
char * federal_marital_status
int federal_withholding_allowances
int federal_withholding_additional_period_amount
boolean state_withholding_exempt
char *state_marital_status
int state_withholding_allowances
int state_itemized_deduction_allowances
int retirement_contribution_plan_employee_period_amount
int retirement_contribution_plan_employer_period_amount
int health_insurance_employee_period_amount
int health_insurance_employer_period_amount
double union_dues_period_amount
char *terminated_date

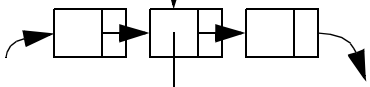
LIST *
employee_work_day_list

LIST *
employee_work_period_list

LIST *
customer_sale_list



CUSTOMER_SALE

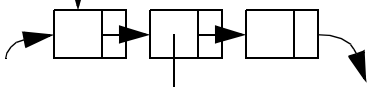


EMPLOYEE_WORK_PERIOD

char *full_name
char *street_address
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
char *transaction_date_time

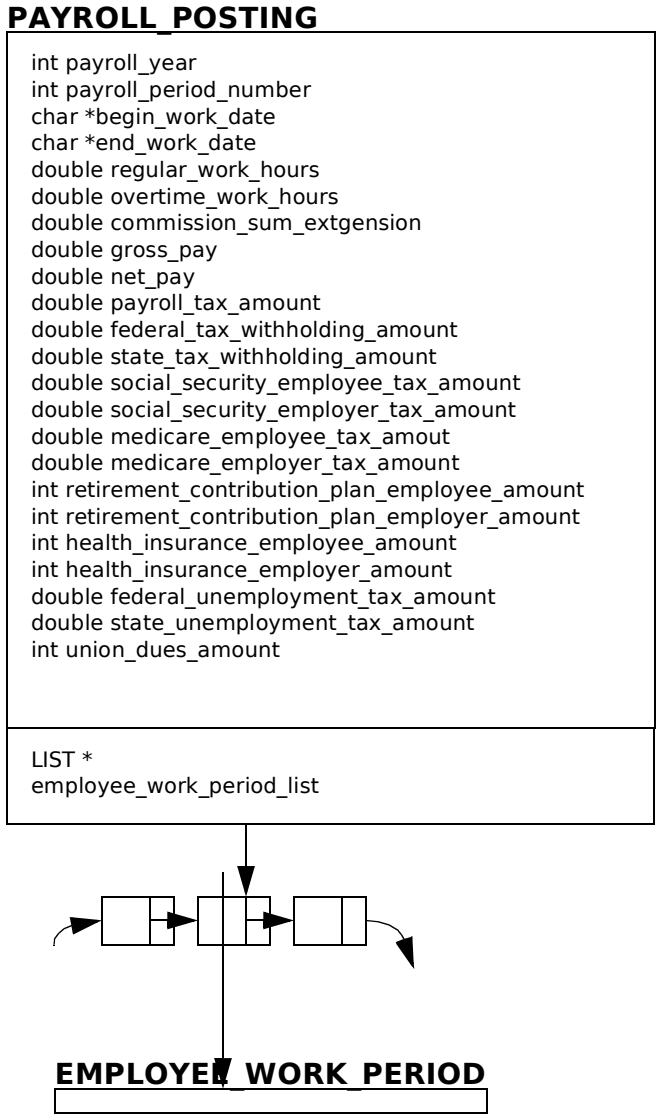
TRANSACTION *
transaction

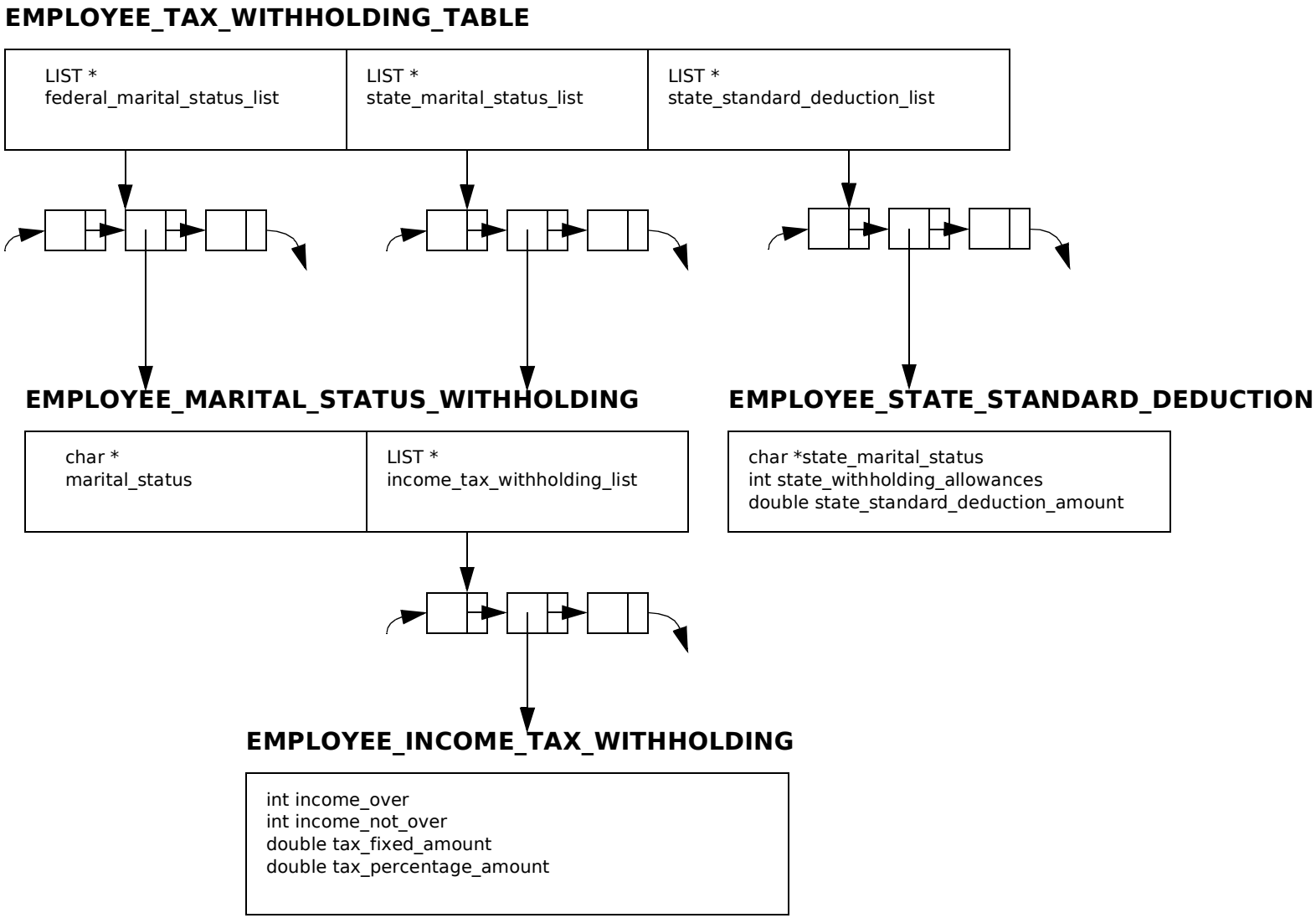
TRANSACTION



EMPLOYEE_WORK_DAY

char *begin_work_date_time
char *end_work_date_time
double employee_work_hours
boolean overtime_work_day


PAYROLL_POSTING
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extgension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
LIST *
employee_work_period_list
EMPLOYEE_WORK_PERIOD


EMPLOYEE_TAX_WITHHOLDING_TABLE
LIST *
federal_marital_status_list
LIST *
state_marital_status_list
LIST *
state_standard_deduction_list
EMPLOYEE_MARITAL_STATUS_WITHHOLDING
char *
marital_status
LIST *
income_tax_withholding_list
EMPLOYEE_STATE_STANDARD_DEDUCTION
char *state_marital_status
int state_withholding_allowances
double state_standard_deduction_amount
EMPLOYEE_INCOME_TAX_WITHHOLDING
int income_over
int income_not_over
double tax_fixed_amount
double tax_percentage_amount

FEEDER_LOAD

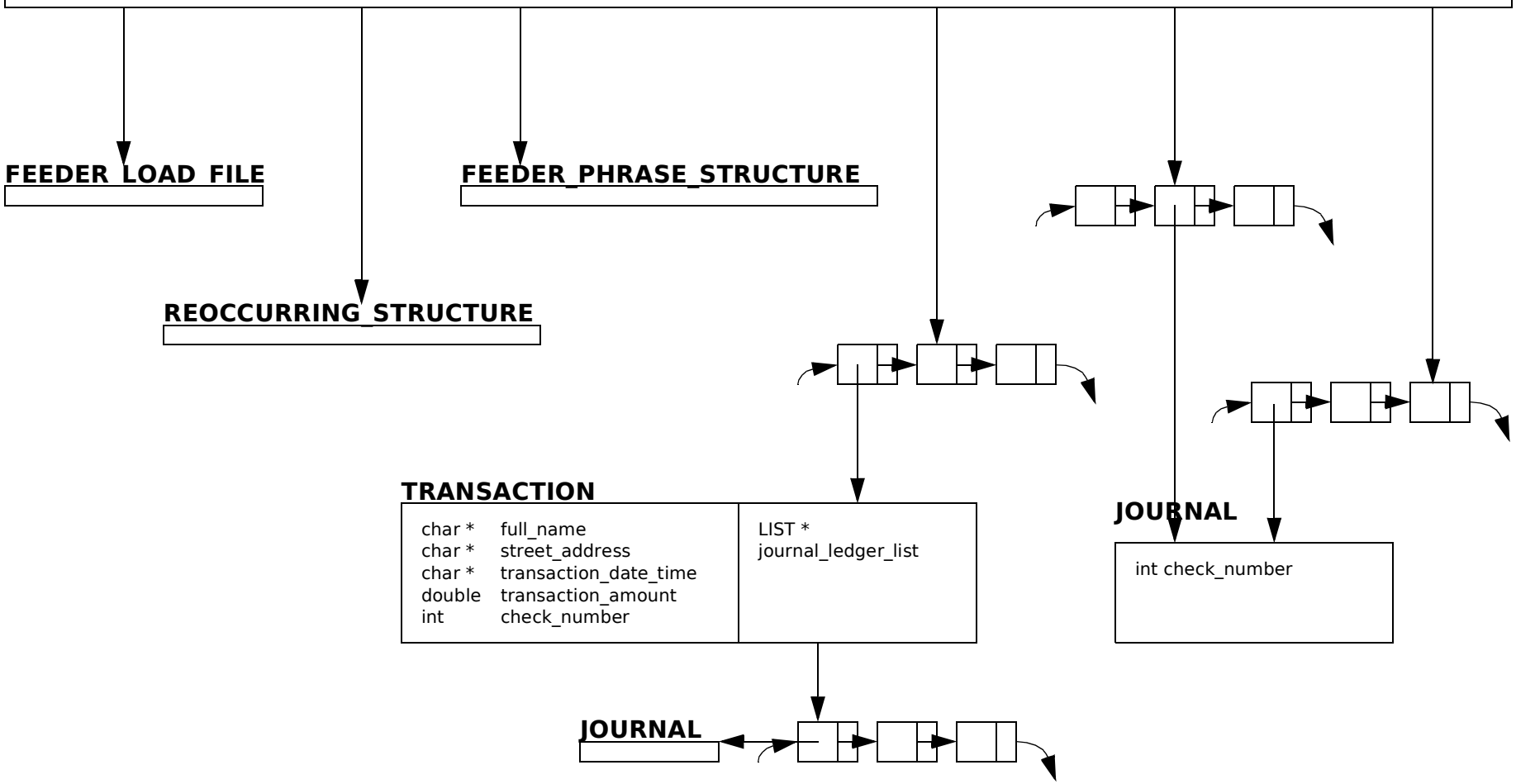
```
/* Usage */
FEEDER_LOAD *feeder_load_new(
    char *process_name,
    char *login_name,
    char *feeder_account,
    char *filename,
    int date_column /* one_based */,
    int description_column /* one_based */,
    int debit_column /* one_based */,
    int credit_column /* one_based */,
    int balance_column /* one_based */,
    boolean reverse_order,
    double bank_closing_balance );
```

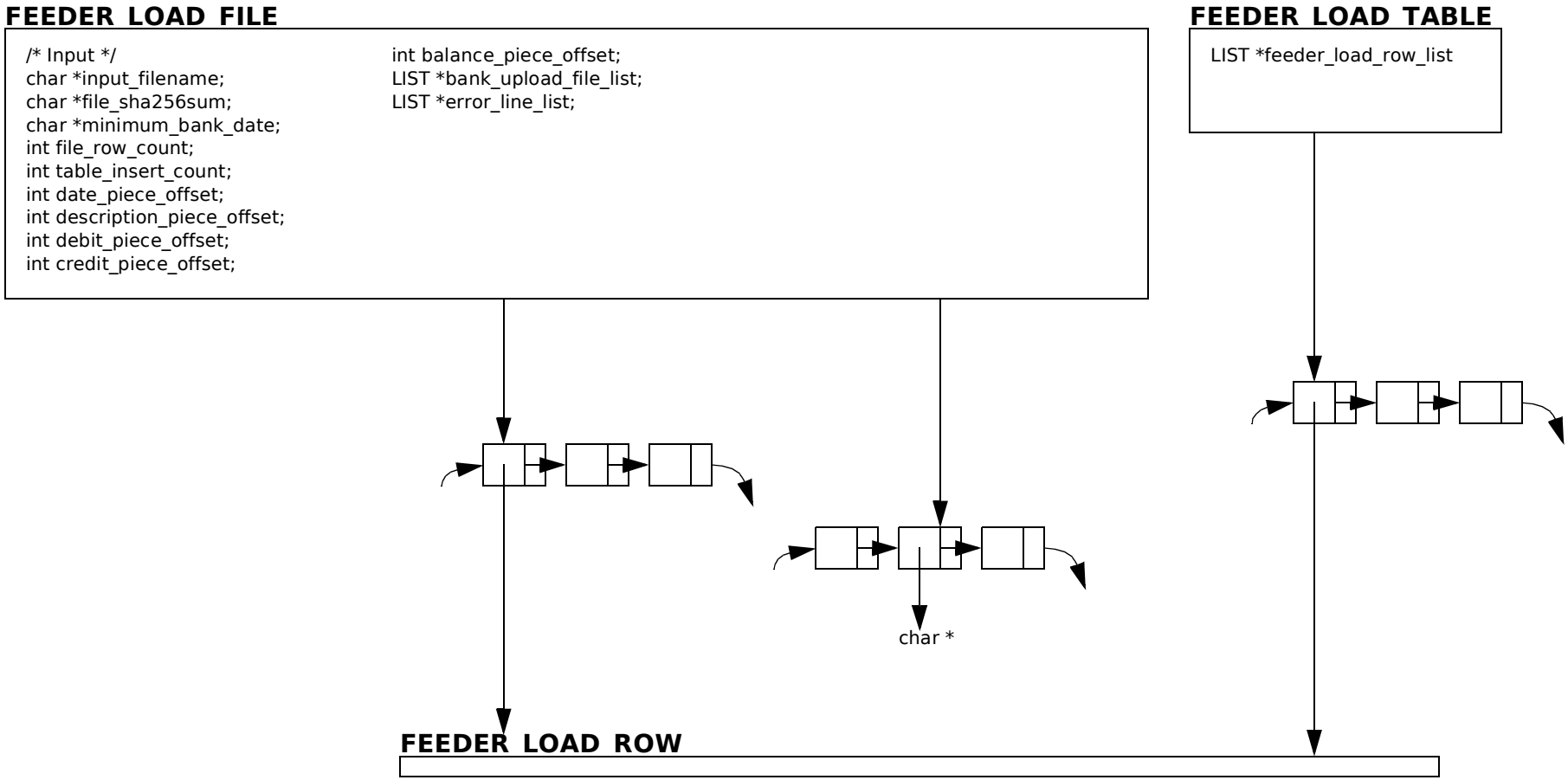
```
/* Process */
FEEDER_LOAD_EVENT *feeder_load_event =
    feeder_load_event_new(
```

FEEDER_LOAD_EVENT

FEEDER_LOAD_EVENT

```
/* Usage */
FEEDER_LOAD_EVENT *feeder_load_event_new(
      char *feeder_account,
      char *input_filename,
      double bank_closing_balance );

/* Input */
char *feeder_account;
char *input_filename;
double bank_ending_balance;

/* Process */
FEEDER_LOAD_FILE *feeder_load_file_fetch(
      char *input_filename );

FEEDER_PHRASE *feeder_phrase_structure_fetch(
      char *feeder_account );

int starting_sequence_number;
char *bank_upload_date_time;
LIST *existing_cash_journal_ledger_list;
LIST *non_reconciled_cash_journal_ledger_list;
LIST *uncleared_checks_transaction_list;
```

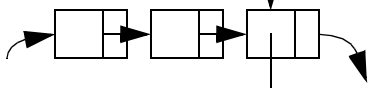



FEEDER_LOAD_FILE
/* Input */
char *input_filename;
char *file_sha256sum;
char *minimum_bank_date;
int file_row_count;
int table_insert_count;
int date_piece_offset;
int description_piece_offset;
int debit_piece_offset;
int credit_piece_offset;
int balance_piece_offset;
LIST *bank_upload_file_list;
LIST *error_line_list;
FEEDER_LOAD_TABLE
LIST *feeder_load_row_list
char *
FEEDER_LOAD_ROW

FEEDER_LOAD_ROW

```
char *bank_date;
char *bank_description;
int sequence_number;
double bank_amount;
double bank_running_balance;
char *fund_name;
int check_number;
boolean existing_bank_upload;
boolean existing_transaction;
LIST *reconciled_transaction_list;
JOURNAL *cleared_journal_ledger;
```

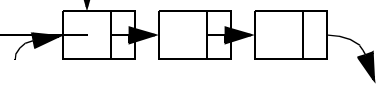
```
JOURNAL *feeder_check_number_existing_journal(
        LIST *existing_cash_journal_list,
        int check_number );

LIST *feeder_match_sum_existing_journal_list(
        LIST *existing_cash_journal_list,
        double abs_bank_amount,
        boolean check_debit );

TRANSACTION *feeder_phrase_match_build_transaction(
        LIST *reoccurring_transaction_list,
        char *bank_date,
        char *bank_description_embedded,
          double abs_bank_amount );
```



TRANSACTION

JOURNAL

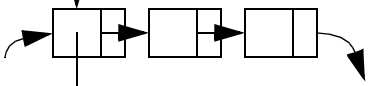
JOURNAL



TRANSACTION

REOCCURRING_STRUCTURE

```
LIST *reoccurring_list;
```



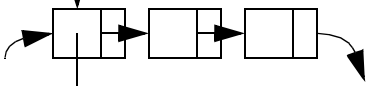
REOCCURRING_TRANSACTION

```
char *full_name
char *street_address
char *transaction_description
char *debit_account
char *credit_account
double transaction_amount
double accrued_daily_amount
double accrued_monthly_amount
char *rental_property_street_address
```

optional

FEEDER_PHRASE_STRUCTURE

```
/* Usage */
FEEDER_PHRASE_STRUCTURE *feeder_structure_new(
      char *bank_account );

/* Process */
LIST *feeder_phrase_list( void );
```



FEEDER_PHRASE

```
char *full_name
char *street_address
char *transaction_description
char *post_account
double transaction_amount
char *feeder_phrase;
```

PAYPAL

```
/* Usage */
PAYPAL *paypal_fetch(
      spreadsheet_filename,
      date_label );

/* Process */
SPREADSHEET *spreadsheet =
      spreadsheet_fetch(
            spreadsheet_filename,
            date_label );
```

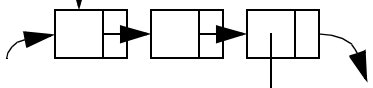
SPREADSHEET

PAYPAL_DATASET

```
/* Input */
char *date_A;
char *time_B;
char *timezone_C;
char *full_name_D;
char *transaction_type_E;
char *transaction_status_F;
char *currency_G;
char *gross_revenue_H;
char *transaction_fee_I;
char *net_revenue_J;
char *from_email_address_K;
char *to_email_address_L;
char *transaction_ID_M;
char *shipping_address_N;
char *address_status_O;
char *item_title_P;
char *item_ID_Q;
char *shipping_handling_amount_R;
char *insurance_amount_S;
char *sales_tax_T;
char *option_1_name_U;
char *option_1_value_V;
char *option_2_name_W;
char *option_2_value_X;
char *reference_txn_ID_Y;
char *invoice_number_Z;
char *custom_number_AA;
char *quantity_AB;
char *receipt_ID_AC;
char *balance_AD;
char *address_line_1_AE;
char *address_2_district_neighborhood_AF;
char *town_city_AG;
char *state_province_AH;
char *zip_code_AI;
char *country_AJ;
char *contact_phone_number_AK;
char *subject_AL;
char *note_AM;
char *country_code_AN;
char *balance_impact_AO;
```

SPREADSHEET

```
/* Input */
char *spreadsheet_filename;
char *date_label;

/* Process */
LIST *spreadsheet_column_list(
      spreadsheet_filename,
      date_label );

char *maximum_date;

char *spreadsheet_minimum_date(
      &maximum_date,
      spreadsheet_filename );

char *spreadsheet_heading_data(
      spreadsheet_column_list(),
      input_row,
      heading );
```



SPREADSHEET_COLUMN

```
/* Usage */
SPREADSHEET_COLUMN *spreadsheet_column_new(
      heading,
      piece_offset );

/* Process */
SPREADSHEET_COLUMN *spreadsheet_column_heading_seek(
      spreadsheet_column_list,
      heading );
```

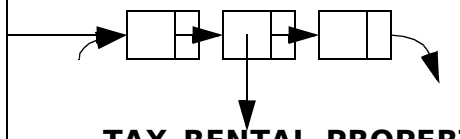
SPREADSHEET_UPLOAD_EVENT

```
/* Input */
char *spreadsheet_filename;

/* Process */
char *spreadsheet_upload_date_time =
      date_time19_now();

char *spreadsheet_upload_file_sha256sum(
      spreadsheet_filename );
```

TAX

```
/* Usage */
TAX *tax_new(
      char *tax_form_string,
      int tax_year );

/* Input */
char *tax_form_string;
int tax_year;

/* Attributes */
```

```
TAX_FORM *tax_form =
      tax_form_fetch();

RECOVERY *recovery;

LIST *tax_rental_property_list =
      tax_rental_property_list_fetch(
            tax_form_string,
            tax_year );
```



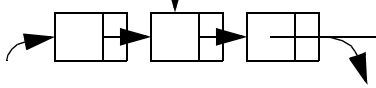
TAX_RENTAL_PROPERTY

TAX_FORM

```
/* Usage */
TAX_FORM *tax_form_fetch(
      char *tax_form_string,
      int tax_year,
      boolean fetch_line_list,
      boolean fetch_account_list,
      boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;

/* Input */
int tax_year;

/* Process */
LIST *tax_form_line_list(
      tax_form_string,
      tax_year,
      fetch_account_list,
      fetch_journal_list );
```

RECOVERY

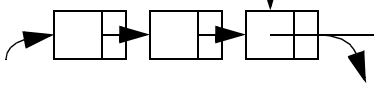
```
double fixed_asset_recovery_amount;
double prior_fixed_asset_recovery_amount;
double property_recovery_amount;
double prior_property_recovery_amount;
double total_recovery_amount;
```



TAX_FORM_LINE

```
/* Usage */
LIST *tax_form_line_list(
      char *tax_form_string,
      int tax_year,
      boolean fetch_account_list,
      boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line_string;
char *tax_form_description;
boolean itemize_accounts;

/* Input */
int tax_year;
```

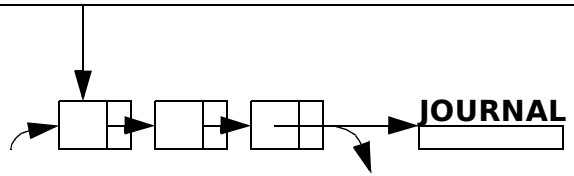
```
/* Process */
LIST *tax_form_line_account_list(
      tax_form_string,
      tax_form_line_string,
      tax_year,
      fetch_journal_list );

/* Operations */
void tax_form_line_list_steady_state(
      tax_form_line_list );

double tax_form_line_total(
      tax_form_line_account_list() );
```



TAX_FORM_LINE_ACCOUNT

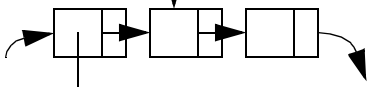
TAX_FORM_LINE_ACCOUNT

```
/* Usage */
LIST *tax_form_line_account_list(
      char *tax_form_string,
      char *tax_form_line,
      int tax_year,
      boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line;
char *account_name;

/* Process */
ACCOUNT *account =
      account_fetch(
            account_name,
            1 /* fetch_subclassification */,
            1 /* fetch_entity */ );

if ( fetch_journal_list )
{
      boolean subclassification_current_liability(
            account->subclassification_name );

      LIST *journal_year_list(
            tax_year,
            account_name,
            1 /* fetch_memo */ );

      double tax_form_line_account_total(
            journal_year_list(),
            account->
                  subclassification->
                  entity->
                  accumulate_debit,
            subclassification_current_liability() );
}
```



TAX_RENTAL_PROPERTY

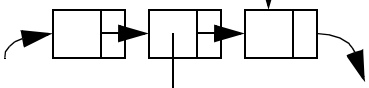
```
/* Usage */
LIST *tax_rental_property_list_fetch(
      char *tax_form_string,
      int tax_year );

/* Attributes */
char  *property_street_address;

/* Process */
LIST *tax_rental_property_string_list(
      tax_rental_property_list );

LIST *tax_form_line_rental_list(
      tax_form_line_list(),
      tax_rental_property_string_list );

double recovery_amount;
double revenue_total [23a];
double mortgage_interest_total [23c];
double depreciation_total [23d];
double expense_total [23e];
double positive_net_income [24];
double loss_total [25];
double tax_rental_property_total();
```



TAX_FORM_LINE_RENTAL

```
char *tax_form_line;
char *tax_form_description;

enum tax_form_line_category;
LIST *tax_form_line_account_list();

LIST *rental_property_list;
double tax_form_line_rental_property_total();
```

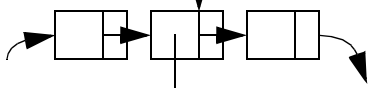
revenue
mortgage_interest_paid
depreciation_expense
operating_expense
loss



TAX_FORM_LINE_ACCOUNT

INVESTMENT_ACCOUNT

```
ENTITY *financial_entity =
      entity_new(
            full_name,
            street_address );

char *account_number;
char *investment_classification;
char *investment_purpose;

LIST *investment_account_balance_list(
full_name,
street_address,
account_number );
```

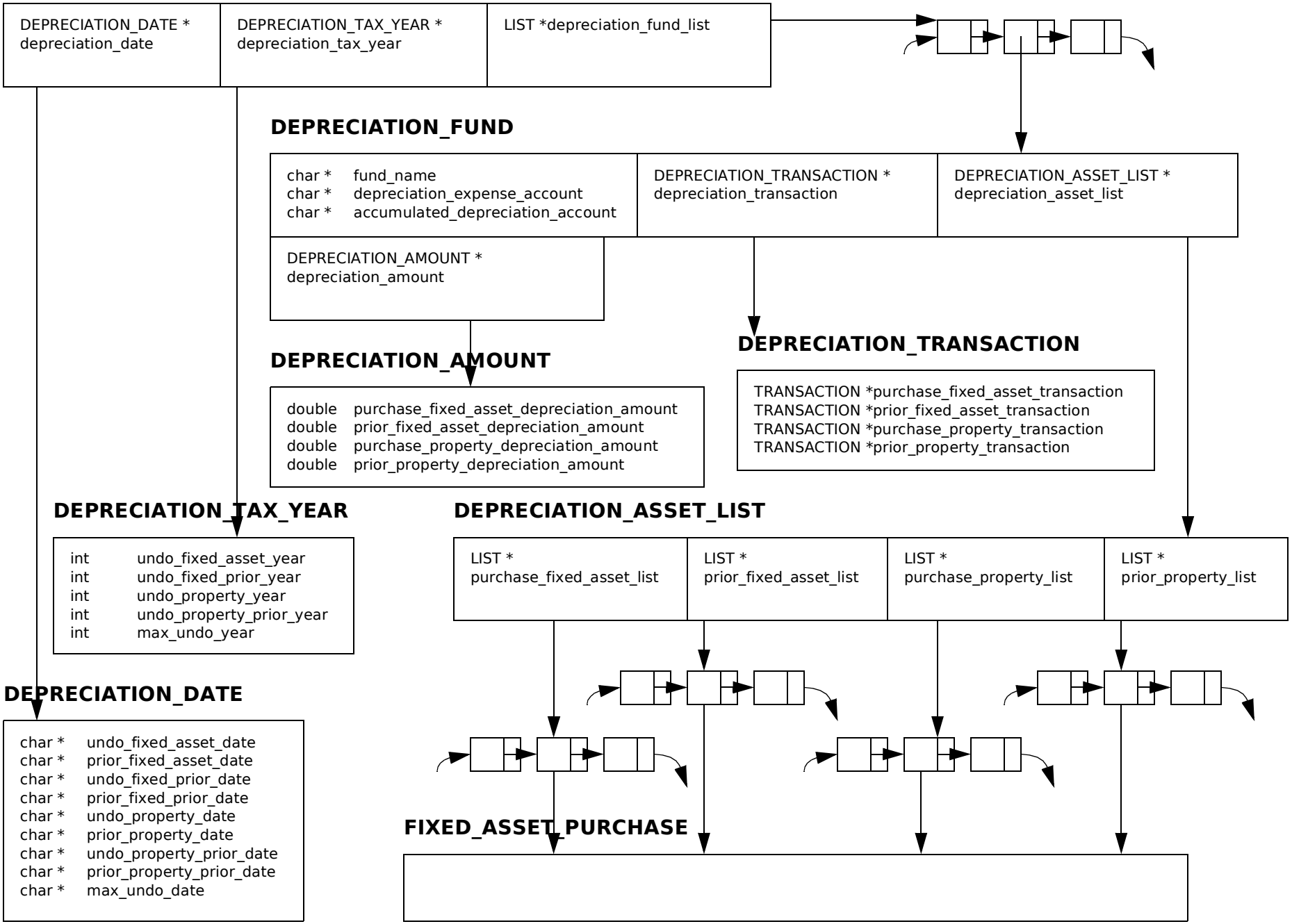


ACCOUNT_BALANCE

```
char *date
double balance;
double balance_change;
int balance_change_percent;
```

TRANSACTION

ACCOUNT_BALANCE_EQUITY

```
char *date_time
char *investment_operation
double share_price
doubel share_quantity_change
double share_quantity_balance
double cash_in
double market_value
double book_value_change
double book_value_balance
double moving_share_price
double unrealized_gain_balance
double unrealized_gain_change
double reaqlized_gain
boolean table_first_row
char *transaction_date_time
TRANSACTION *
transaction
```

DEPRECIATION_STRUCTURE